

Prototipo 4 — Atributos De Calidad, Parte 2

Arquitectura de Software — 2026-I Universidad Nacional de Colombia

1. Equipo

Nombre del Equipo: 2B

Nombre Completo
Ana María González Hernández
Iván Daniel Silva Oyola
Daniel Alejandro Ortiz Velosa
Carlos Julián Reyes Piraligua
Sergio Esteban Rendón Umbarila
Sergio Alejandro Ruiz Hurtado
Cristian Javier Medina Barrios
Camilo Andrés Liévano Rendón

2. Sistema de Software

Nombre

UN Wheels

Logo



Descripción

UN Wheels es una plataforma de movilidad compartida diseñada para la comunidad universitaria de la Universidad Nacional de Colombia. Conecta estudiantes conductores con estudiantes pasajeros dentro de un ecosistema cerrado y verificado mediante correo institucional (@unal.edu.co), centralizando la

publicación y gestión de rutas compartidas que hoy ocurren de forma dispersa en plataformas externas como WhatsApp o grupos de Facebook.

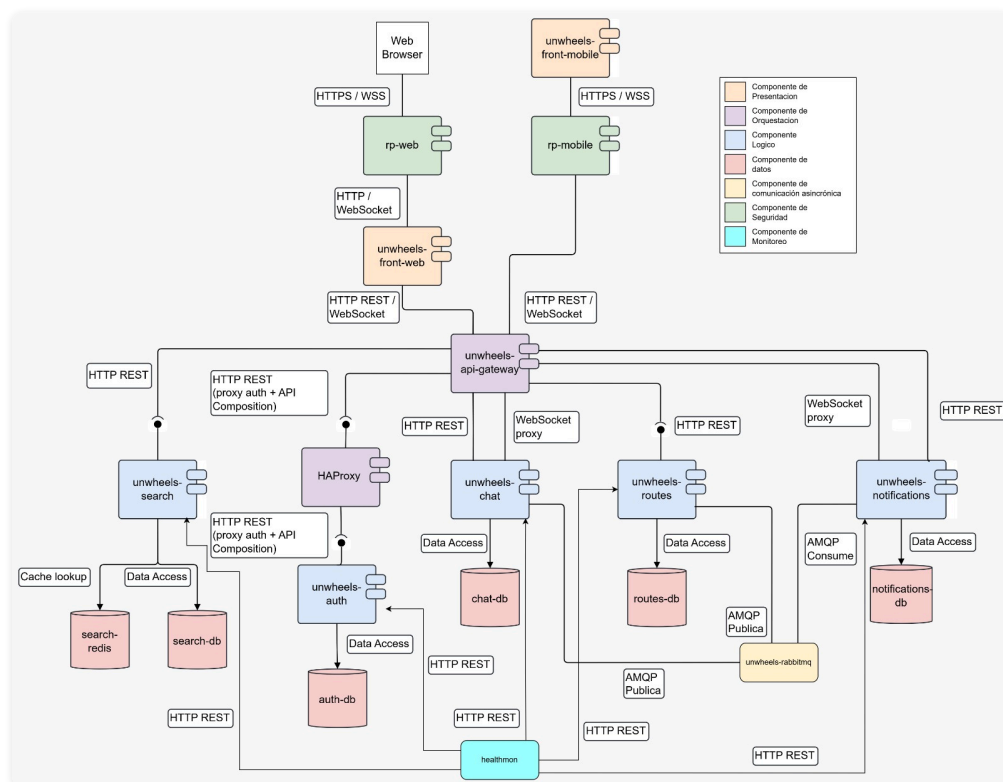
El sistema permite a los conductores publicar rutas con horario, paradas, precio por asiento y cupos disponibles, mientras que los pasajeros pueden buscar rutas, reservar cupos y comunicarse directamente con el conductor mediante chat en tiempo real. El modelo se basa en un **usuario único con roles dinámicos**: cualquier estudiante actúa como pasajero por defecto, y al registrar un vehículo adquiere la capacidad de conductor. Las reservas incluyen control de concurrencia para garantizar que no se sobrevendan los cupos.

La arquitectura sigue un patrón de microservicios distribuidos con cinco lenguajes de programación de propósito general distintos (TypeScript, Python, JavaScript, Go, Dart), cinco componentes de tipo lógico, un API Gateway para el tráfico norte-sur, un broker de mensajes (RabbitMQ) para la comunicación este-oeste y la orquestación asíncrona entre servicios, y cinco componentes de tipo datos que incluyen bases de datos relacionales y NoSQL. Todo el sistema se despliega como contenedores Docker orquestados mediante Docker Compose.

3. Estructuras Arquitectónicas

3.1 Estructura de Componentes y Conectores (C&C)

Vista C&C



Descripción de Elementos Arquitectónicos y Relaciones

Componentes

Capa Edge

Es la única zona expuesta al mundo exterior (Internet/Host). Aloja los Reverse Proxies, que fungen simultáneamente como enrutadores y firewalls perimetrales (ModSecurity + OWASP CRS embebido).

Componente	Puerto	Descripción
rp-web	8080	Reverse proxy dedicado al cliente web. Enruta el tráfico de páginas hacia el frontend y las conexiones WebSocket directamente al API Gateway. Posee políticas de seguridad flexibles adaptadas a navegadores (ej. permite cargas de hasta 10 MB y admite contenido multipart/form-data para subida de imágenes) e implementa Rate Limiting.
rp-mobile	8081	Reverse proxy dedicado a la aplicación móvil. Enruta el tráfico exclusivamente hacia el API Gateway. Posee un perfil de seguridad sumamente estricto, diseñado para una API pura (límite de carga de 5 MB y aceptación exclusiva de peticiones en formato JSON).

Capa Web-Zone

Componente	Tecnología	Lenguaje	Descripción
unwheels-frontweb	NextJS	TypeScript	Aplicación encargada del renderizado del lado del servidor (SSR) y de la interfaz de usuario web. Reside en una red aislada a la que solo el rp-web tiene acceso, impidiendo la comunicación directa desde cualquier otra fuente.

Capa Gateway-Zone

Componente	Tecnología	Lenguaje	Descripción
unwheels-api-gateway	NodeJS	TypeScript	Orquestador principal de la API. Coordina, enruta y consolida las peticiones recibidas hacia los microservicios correspondientes. Recibe tráfico validado directamente desde rp-mobile, solicitudes internas desde el unwheels-frontweb, y conexiones WebSocket enviadas por el rp-web.

Capa Balance-Zone

Componente	Tecnología	Lenguaje	Descripción
loggueo-lb	HAProxy	—	Load balancer de software que distribuye el tráfico de autenticación entre las tres réplicas del unwheels-auth (loggueo_1, loggueo_2, loggueo_3). Implementa el patrón Load Balancer con algoritmo Round-Robin por defecto, intercambiable por Least Connections o IP Hash mediante configuración en volumen. Realiza health checks activos HTTP (GET /) cada 5 segundos sobre cada réplica, retirando automáticamente del pool toda instancia que acumule 3 fallos consecutivos y reincorporándola tras 2 checks exitosos. Mantiene el nombre DNS loggueo-service, garantizando transparencia total hacia el api-gateway.

Capa Services-Zone

Componente	Tecnología	Lenguaje	Puerto	Descripción
unwheels-auth	FastAPI + SQLAlchemy + Pydantic v2 + Uvicorn	Python 3.x	8000	Autenticación, registro de usuarios (validando dominio <code>@unwheels.edu.co</code>), emisión y validación de JWT, y gestión de perfiles y vehículos. Es el único servicio que emite tokens JWT.
unwheels-routes	Express.js + Mongoose	JavaScript (Node.js)	4000	Gestión del ciclo de vida completo de rutas y reservas: creación, consulta, control de concurrencia para cupos, y transiciones de estado de reservas (PENDING → CONFIRMED / REJECTED). Publica eventos de dominio en RabbitMQ al completar operaciones de negocio.
unwheels-chat	NestJS + Socket.IO + Mongoose	TypeScript (Node.js)	3001	Mensajería bidireccional en tiempo real entre conductores y pasajeros. Expone un Gateway Socket.IO para mensajería en tiempo real y un Controller HTTP para consultar historial y listar conversaciones. Publica eventos <code>chat.message</code> en RabbitMQ.
unwheels-notifications	NestJS + Socket.IO + Mongoose	TypeScript (Node.js)	3002	Consumidor de eventos asíncronos de RabbitMQ. Persiste notificaciones en MongoDB y las entrega en tiempo real vía WebSocket (Socket.IO, namespace <code>/notifications</code>). Actúa como puente AMQP ↔ WebSocket dentro del mismo proceso.
unwheels-search	Gin + pgx/v5	Go	6000	Búsqueda geoespacial de rutas disponibles. Mantiene un índice de lectura optimizada en PostgreSQL+PostGIS, sincronizado periódicamente con unwheels-routes mediante polling HTTP. Expone un único endpoint público que acepta consultas de texto libre y radio geográfico.

Componente	Tecnología	Lenguaje	Descripción
healthmon	Node.js	JavaScript	Monitor centralizado de salud del sistema. Implementa el patrón Health Check / Heartbeat enviando probes HTTP periódicos (cada 15 s) a los cinco microservicios de lógica de negocio y al load balancer HAProxy. Mantiene contadores de fallos consecutivos por servicio y emite eventos estructurados en JSON (UNHEALTHY / RECOVERED) a stdout cuando un servicio supera el umbral de 3 fallos consecutivos. Expone los endpoints GET /status (estado agregado de todos los servicios) y GET /healthz (liveness del propio monitor).

Componente de Orquestación

Componente	Tecnología	Descripción
unwheels-rabbitmq	RabbitMQ 3.13 (puertos 5672 AMQP / 15672 Management UI)	Broker de mensajes para comunicación asíncrona este-oeste entre microservicios. Utiliza un topic exchange durable (<code>unwheels.events</code>) con routing keys para enrutamiento de eventos. Habilita el patrón Publish-Subscribe con desacoplamiento total entre productores y consumidores.

Capa Data-Zone

Componente	Tipo	Usado por	Descripción
auth-db	PostgreSQL 16 (Relacional)	unwheels-auth	Almacena usuarios y vehículos con integridad referencial. Transacciones ACID. Accedido mediante SQLAlchemy ORM.
routes-db	MongoDB 7 (Documental)	unwheels-routes	Almacena rutas, reservas y reglas de disponibilidad (SPECIFIC_DATES, WEEKLY_RECURRENCE) con esquemas flexibles. Accedido mediante Mongoose.
chat-db	MongoDB 7 (Documental)	unwheels-chat	Almacena conversaciones y mensajes en formato append-only. Accedido mediante Mongoose.
notifications-db	MongoDB 7 (Documental)	unwheels-notifications	Almacena historial de notificaciones con TTL de 30 días e índice compuesto por destinatario. Accedido mediante Mongoose.
search-db	PostgreSQL 16 + PostGIS 3.4 (Geoespacial)	unwheels-search	Proyección de lectura de rutas con columnas GEOGRAPHY para consultas espaciales eficientes (<code>ST_DWithin</code> , <code>ST_Distance</code>). Accedido mediante pgx/v5.
search-redis	Redis 7	unwheels-auth	Caché en memoria dedicada al unwheels-search-service. Implementa el patrón Cache-Aside almacenando resultados de búsquedas geoespaciales con un TTL de 120 segundos y una política de evicción allkeys-lru con límite de 128 MB. Permite que búsquedas repetidas con los mismos parámetros se resuelvan en memoria sin ejecutar queries sobre PostGIS, reduciendo la latencia de respuesta y la carga sobre search-db durante picos de demanda.

Conectores (Relaciones)

Conectores HTTP (Cliente ↔ Gateway ↔ Servicios)

Conector	Tipo	Protocolo	Endpoints	Dirección
Cliente Web → Reverse Proxy Web (WAF)	Llamada a Procedimiento	HTTPS	<code>security/reverse-proxies/rp-web/</code>	Bidireccional: Cliente (Navegador Web) → rp-web
Cliente Mobile → Reverse Proxy Mobile (WAF)	Llamada a Procedimiento	HTTPS	<code>security/reverse-proxies/rp-mobile/</code>	Bidireccional: Cliente (App Mobile) → rp-mobile
Cliente Web → Gateway (WebSocket bypass)	Evento	WSS	<code>/api/</code>	Bidireccional: Cliente (Navegador Web) → Gateway
Reverse Proxy Web → Frontend Web	Proxy Interno	HTTP	<code>security/reverse-proxies/rp-web/</code>	Unidireccional: rp-web → Frontend SSR Container
Frontend Web → API Gateway	Llamada a Procedimiento	HTTP REST	<code>/api/</code>	Bidireccional: Frontend → Gateway
Reverse Proxy Mobile → API Gateway	Proxy Seguro	HTTP REST	<code>security/reverse-proxies/rp-mobile/</code> , <code>/api/</code>	Bidireccional: rp-mobile → Gateway
REST (Auth)	Llamada a Procedimiento	HTTP/JSON	<code>/api/auth/*</code>	Bidireccional: Cliente → Gateway → unwheels-auth
REST (Routes)	Llamada a Procedimiento	HTTP/JSON	<code>/api/routes/*</code> , <code>/api/reservations/*</code> , <code>/api/vehicles/*</code>	Bidireccional: Cliente → Gateway → unwheels-routes
REST (Notifications)	Llamada a Procedimiento	HTTP/JSON	<code>/api/notifications/*</code>	Bidireccional: Cliente → Gateway → unwheels-notifications
REST (Search)	Llamada a Procedimiento	HTTP/JSON	<code>/api/search/*</code>	Bidireccional: Cliente → Gateway → unwheels-search
REST (Chat HTTP)	Llamada a Procedimiento	HTTP/JSON	<code>/api/chat/*</code>	Bidireccional: Cliente → Gateway → unwheels-chat
WebSocket/Socket.IO (Chat)	Evento	WS + Socket.IO	<code>/api/chat/socket.io</code>	Bidireccional: Cliente ↔ Gateway (proxy) ↔ unwheels-chat (namespace <code>/</code>)
WebSocket/Socket.IO (Notif.)	Evento	WS + Socket.IO	<code>/api/notifications/socket.io</code>	Servidor → Cliente: Gateway (proxy) ↔ unwheels-notifications (namespace <code>/notifications</code>)

Conectores de Message Broker (Servicio ↔ RabbitMQ)

Conector	Tipo	Protocolo	Routing Keys	Publicador → Consumidor
AMQP (Eventos de Reserva)	Unidireccional: Evento (Pub/Sub)	AMQP 0.9.1	<code>reservation.requested</code> , <code>reservation.accepted</code> , <code>reservation.rejected</code>	unwheels-routes → unwheels-rabbitmq → unwheels-notifications
AMQP (Eventos de Ruta)	Unidireccional: Evento (Pub/Sub)	AMQP 0.9.1	<code>route.deleted</code>	unwheels-routes → unwheels-rabbitmq → unwheels-notifications
AMQP (Eventos de Chat)	Unidireccional: Evento (Pub/Sub)	AMQP 0.9.1	<code>chat.message</code>	unwheels-chat → unwheels-rabbitmq → unwheels-notifications

Conectores de Acceso a Datos

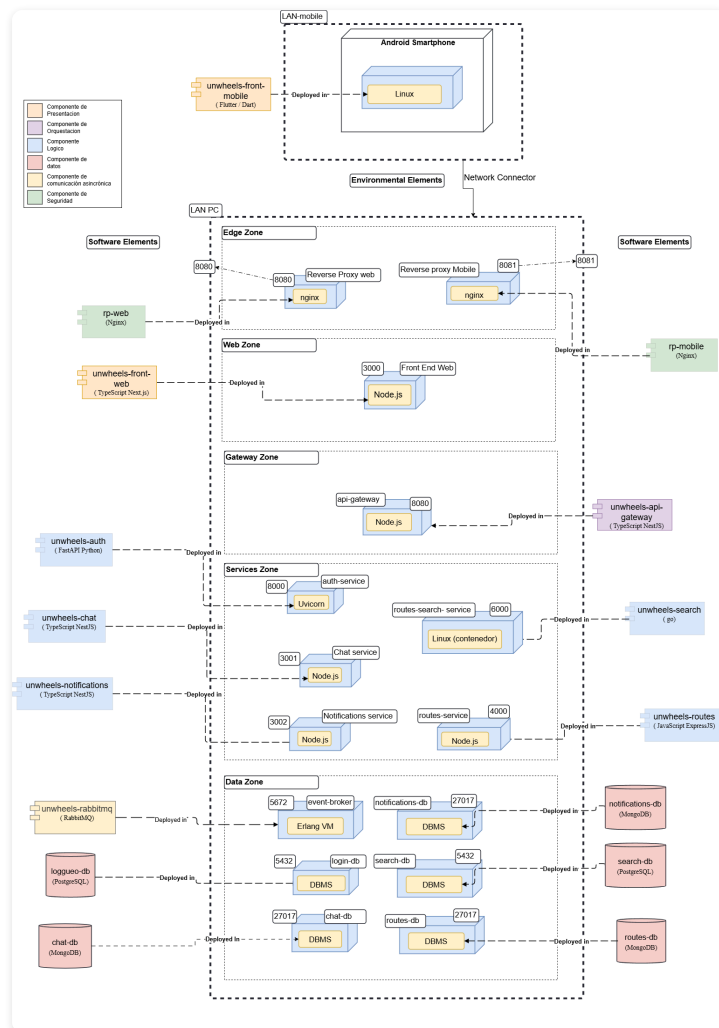
Conector	Tipo	Tecnología	Componentes
db_connector_relational	Acceso a Datos	SQLAlchemy (Python)	unwheels-auth → auth-db
db_connector_documental (routes)	Acceso a Datos	Mongoose (Node.js)	unwheels-routes → routes-db
db_connector_documental (chat)	Acceso a Datos	Mongoose (Node.js)	unwheels-chat → chat-db
db_connector_documental (notif.)	Acceso a Datos	Mongoose (Node.js)	unwheels-notifications → notifications-db
db_connector_geoespacial	Acceso a Datos	pgx/v5 (Go)	unwheels-search → search-db
http_polling (Search Sync)	Llamada a Procedimiento	HTTP/JSON	unwheels-search → unwheels-routes (sincronización periódica del índice PostGIS)

Descripción de Estilos y Patrones Arquitectónicos

Estilo / Patrón	Aplicación en UN Wheels
Microservicios	Cinco componentes lógicos independientes (auth, routes, chat, notifications, search), cada uno con su propio código fuente, runtime y datastore dedicado. Los servicios son desplegables de forma independiente y usan la tecnología óptima para su dominio.
API Gateway	NestJS actúa como punto de entrada único para todo el tráfico externo (norte-sur). Enruta peticiones, centraliza la validación JWT, gestiona cookies HttpOnly y realiza proxy transparente de WebSocket. Los clientes no conocen la topología interna.
Publish-Subscribe	RabbitMQ habilita la comunicación asíncrona (tráfico este-oeste) entre microservicios. Los servicios productores publican eventos de dominio en el topic exchange <code>uniwheels.events</code> ; el consumidor (notifications) se suscribe con bindings comodín. Esto desacopla completamente productores de consumidores.
Bridge AMQP↔WebSocket	unwheels-notifications actúa como puente entre dos protocolos: consume eventos AMQP del broker y los entrega en tiempo real vía Socket.IO al cliente. Ambas interfaces conviven en el mismo proceso, reduciendo la latencia a $\approx 10\text{--}15$ ms entre cola y navegador.
Server-Side Rendering (SSR)	El frontend web usa Next.js 15 para renderizar páginas en el servidor. El middleware SSR valida la cookie HttpOnly antes de renderizar rutas protegidas, sin exponer el JWT al navegador.
Database per Service	Cada microservicio posee sus propios datos en una base de datos independiente: auth → PostgreSQL, routes / chat / notifications → MongoDB ×3, search → PostgreSQL+PostGIS. No existe compartición directa de bases de datos entre servicios.
SOFEA	El backend nunca genera HTML; solo sirve JSON. Esto permite que el frontend web y el frontend móvil evolucionen de forma independiente sobre la misma superficie de API sin duplicar lógica de backend.

3.2 Estructura de Despliegue

Vista de Despliegue



Descripción de Elementos Arquitectónicos y Relaciones

El sistema se despliega en un clúster distribuido multi-nodo utilizando Docker Swarm sobre una red local. La infraestructura se compone de dos nodos físicos: un Nodo Manager, que centraliza el plano de control, la persistencia y la suite perimetral, y un Nodo Worker, configurado como nodo de computación dedicado para la alta disponibilidad del core de negocio. La arquitectura mantiene su estrategia de Network Segmentation mediante redes virtuales superpuestas (Overlay Networks de Swarm), aislando el tráfico en cinco zonas lógicas:

Zona Móvil (LAN-mobile)

Elemento de Despliegue	Runtime	Descripción
unwheels-front-mobile	Flutter / Dart VM sobre Android (Linux)	Aplicación nativa ejecutada en el smartphone del usuario. Se comunica con el backend a través del reverse proxy móvil (rp-mobile) por HTTP/REST y WebSocket en el puerto 8081.

Edge Zone (puertos expuestos al host)

Elemento de Despliegue	Contenedor	Runtime	Puerto	Descripción
rp-web	reverse-proxy-web	Nginx	8080 (host) → 8080 (contenedor)	Punto de entrada del tráfico web. Recibe peticiones del navegador y las enruta hacia el frontend web (puerto 3000) o hacia el api-gateway (puerto 8080 interno).
rp-mobile	reverse-proxy-mobile	Nginx	8081 (host) → 8081 (contenedor)	Punto de entrada exclusivo del tráfico móvil. Recibe peticiones de la app Flutter y las enruta hacia el api-gateway.

Web Zone (red interna Docker)

Elemento de Despliegue	Contenedor	Runtime	Puerto (interno)	Descripción
unwheels-front-web	web-app	Node.js (Next.js standalone)	3000	Servidor Next.js SSR. Accesible desde el exterior únicamente a través de rp-web.

Gateway Zone (red interna Docker)

Elemento de Despliegue	Contenedor	Runtime	Puerto (interno)	Descripción
unwheels-api-gateway	api-gateway	Node.js (NestJS)	8080	Enrutador central de tráfico backend. Valida JWT, enruta peticiones a los microservicios de negocio y gestiona conexiones WebSocket. Solo accesible desde los reverse proxies de la Edge Zone.

Services Zone (red interna Docker)

Elemento de Despliegue	Contenedor	Runtime	Puerto (interno)	Despliegue en Clúster (Ubicación)
unwheels-auth	auth-service	Python 3.x / Uvicorn (FastAPI)	8000	1 Instancia (Fija en Nodo Manager)
unwheels-routes	routes-service	Node.js (Express)	4000	Cluster Pattern (2 Réplicas): 1 Instancia en Nodo Manager y 1 Instancia en Nodo Worker.
unwheels-chat	chat-service	Node.js (NestJS)	3001	1 Instancia (Fija en Nodo Manager)
unwheels-notifications	notifications-service	Node.js (NestJS)	3002	1 Instancia (Fija en Nodo Manager)
unwheels-search	routes-search-service	Go / Gin (contenedor Linux)	6000	1 Instancia (Fija en Nodo Manager)

Data Zone (red interna Docker)

Elemento de Despliegue	Contenedor	Runtime	Puerto (interno)
unwheels-rabbitmq	<code>event-broker</code>	RabbitMQ 3.13 (Erlang VM)	5672 (AMQP)
auth-db	<code>auth-db</code>	PostgreSQL 16	5432
search-db	<code>search-db</code>	PostgreSQL 16 + PostGIS	5432
chat-db	<code>chat-db</code>	MongoDB 7	27017
notifications-db	<code>notifications-db</code>	MongoDB 7	27017
routes-db	<code>routes-db</code>	MongoDB 7	27017

Relaciones de Despliegue

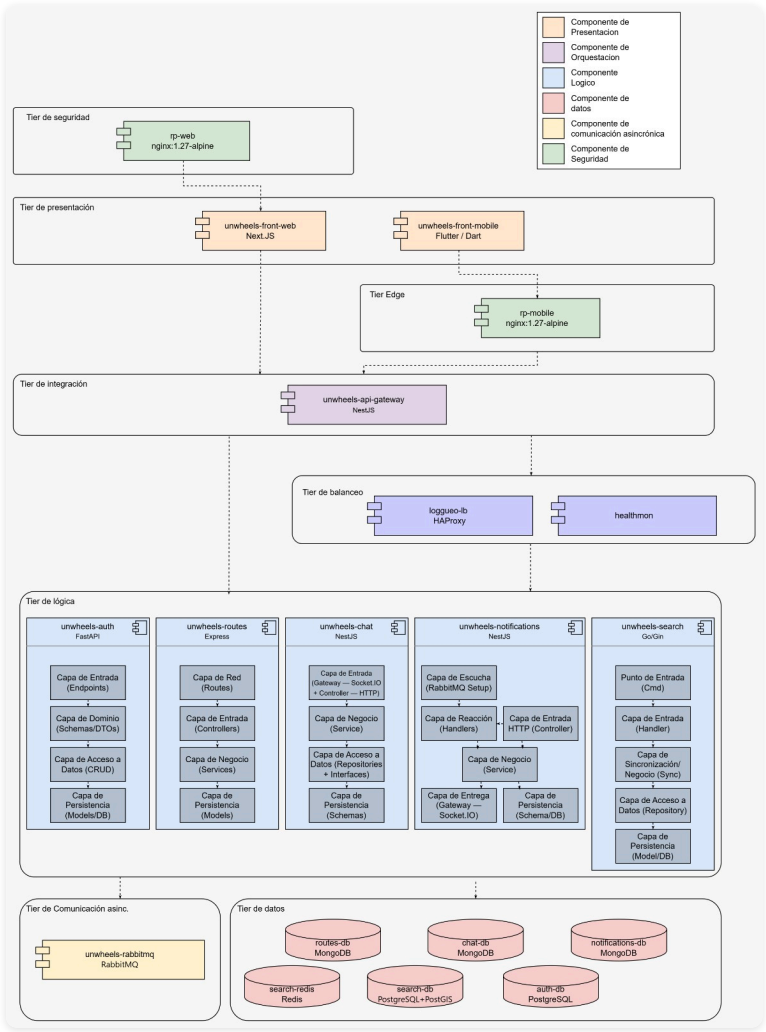
- El tráfico web ingresa por `rp-web` (puerto 8080 del host), que lo enruta hacia `web-app` (puerto 3000) o hacia `api-gateway` (puerto 8080 interno) según la ruta.
- El tráfico móvil ingresa exclusivamente por `rp-mobile` (puerto 8081 del host), que lo enruta hacia `api-gateway`.
- El `api-gateway` no está expuesto directamente al host; solo es alcanzable desde los contenedores de la Edge Zone.
- Todos los contenedores de las zonas internas (Web, Gateway, Services y Data) se comunican a través de la red bridge Docker mediante DNS interno basado en nombres de contenedor.
- El `api-gateway` depende de que `auth-service`, `chat-service`, `routes-service` y `notifications-service` estén saludables antes de iniciar (`depends_on: condition: service_healthy`).
- Los proxies inversos (`rp-web`, `rp-mobile`) dependen de que `api-gateway` esté saludable.
- Los microservicios de negocio dependen de sus respectivos datastores y, cuando aplica, de `event-broker`.
- Los datos persistentes se almacenan en volúmenes Docker: `mongodb_data`, `rabbitmq_data` y `postgres_data`.

Descripción de Patrones Arquitectónicos

Patrón	Aplicación
Containerización	Cada componente se empaqueta como una imagen Docker con su propio Dockerfile. Garantiza despliegues reproducibles, aislados y portables entre entornos.
Dual Reverse Proxy (Edge Zone)	Dos instancias Nginx separan el canal web (puerto 8080) del canal móvil (puerto 8081) como únicos puntos de entrada al sistema. Esto permite políticas de enrutamiento, TLS y rate-limiting diferenciadas por tipo de cliente sin exponer directamente ningún servicio de negocio.
Aislamiento de Red Interna	La red bridge Docker <code>unwheels_default</code> aísla los servicios internos. Los proxies de la Edge Zone son los únicos contenedores con puertos publicados al host; el resto solo hace <code>expose</code> .
Database Service	Cada microservicio tiene su propia instancia de base de datos en un contenedor independiente con su propio volumen Docker. Ningún par de servicios comparte base de datos, garantizando acoplamiento débil en la capa de datos.
Heartbeat Pattern (equipo)	<code>healthmon</code> hace polling cada 15s con umbral de 3 fallos a servicios sin LB propio (chat, routes, notifications, search, liveness del LB). No repite httpchk de réplicas auth — eso lo hace HAProxy. Docker healthcheck usa <code>/healthz</code> (liveness) en réplicas.
Service Discovery Pattern	DNS estable <code>loggueo-service</code> → HAProxy (<code>loggueo-lb</code>) con pool de 3 réplicas. <code>option httpchk GET /readyz</code> (cada 5 s) es la única fuente de verdad para enrutamiento; excluye instancias caídas. Heartbeat observa servicios sin LB, sin duplicar checks a réplicas auth.
Load Balancer Pattern	HAProxy balancea <code>loggueo_1/2/3</code> con Round-Robin (configurable a Least-Conn o Source IP). El api-gateway resuelve una sola URL; el LB excluye réplicas unhealthy vía httpchk.
Despliegue Multi-nodo (Clúster)	la infraestructura lógica se distribuye en un clúster multi-nodo gestionado por Docker Swarm sobre una red local. La suite perimetral, el API Gateway, los servicios de soporte y el Tier de Datos (Stateful) se anclan de forma estricta al Nodo Manager. En contraste, el microservicio crítico de rutas <code>unwheels-routes</code> implementa el Cluster Pattern en redundancia activa dispersando de forma asimétrica una de sus réplicas hacia un Nodo Worker.

3.3 Estructura en Capas

Vista en Capas



Descripción de elementos arquitectónicos y relaciones

La vista organiza el sistema en 8 tiers, desde el punto de entrada público hasta la persistencia de datos:

Tier	Componente(s)	Tecnología	Responsabilidad
Seguridad	<code>rp-web</code>	nginx:1.27-alpine	Único punto de entrada público para tráfico web; combina reverse proxy y WAF.
Presentación	<code>unwheels-front-web</code> , <code>unwheels-front-mobile</code>	Next.js, Flutter/Dart	Clientes web (SSR) y móvil; se comunican con el backend a través del API Gateway.
Edge	<code>rp-mobile</code>	nginx:1.27-alpine	Reverse proxy dedicado al tráfico móvil; oculta el API Gateway del exterior.
Integración	<code>unwheels-api-gateway</code>	NestJS	Punto de entrada unificado del backend; enruta peticiones y aplica políticas de autenticación/autorización.
Balanceo	<code>loggueo-lb</code> , <code>healthmon</code>	HAProxy, Node.js	Capa de equilibrio y monitoreo transversal: HAProxy distribuye tráfico hacia <code>unwheels-auth</code> y <code>healthmon</code> ejecuta heartbeat y observabilidad sobre los microservicios.
Lógica	<code>unwheels-auth</code> , <code>unwheels-routes</code> , <code>unwheels-chat</code> , <code>unwheels-notifications</code> , <code>unwheels-search</code>	FastAPI, Express, NestJS ×2, Go/Gin	Microservicios de negocio, cada uno con arquitectura de capas interna propia.
Comunicación Asíncrona	<code>unwheels-rabbitmq</code>	RabbitMQ	Broker de mensajes que desacopla al productor (<code>unwheels-routes</code>) del consumidor (<code>unwheels-notifications</code>).
Datos	<code>auth-db</code> , <code>routes-db</code> , <code>chat-db</code> , <code>notifications-db</code> , <code>search-db</code> , <code>search-redis</code>	PostgreSQL, MongoDB ×3, PostgreSQL+PostGIS, Redis 7	Persistencia por servicio y caché auxiliar; <code>search-redis</code> complementa la capa de datos para consultas y lectura acelerada.

Estructura interna de capas — Tier de Balanceo:

Componente	Capa	Responsabilidad
<code>loggueo-lb</code>	Load Balancer	Distribuye el tráfico de autenticación entre las tres réplicas de <code>unwheels-auth</code> .
<code>healthmon</code>	Heartbeat / Monitor	Ejecuta probes periódicos y emite eventos de salud para los microservicios.

Estructura interna de capas — Tier de Lógica:

Servicio	Capa	Responsabilidad
unwheels-auth (FastAPI)	Endpoints	Expone los endpoints REST para login, validación de token y gestión de usuarios.
	Schemas/DTOs	Define esquemas de datos y objetos de transferencia entre capas.
	CRUD	Operaciones de lectura/escritura sobre auth-db (PostgreSQL).
	Models/DB	Modelos de datos y conexión directa con la base de datos.
unwheels-routes (Express)	Routes	Define las rutas HTTP y dirige peticiones a los controladores.
	Controllers	Recibe y valida peticiones HTTP, delega a la capa de servicios.
	Services	Lógica de negocio para rutas, control de cupos y reservas.
	Models	Modelos de datos y conexión con routes-db (MongoDB).
unwheels-chat (NestJS)	Socket.IO + HTTP	Gestiona conexiones WebSocket entrantes y peticiones HTTP.
	Service	Lógica de negocio del chat: envío, recepción y gestión de mensajes.
	Repositories + Interfaces	Abstrae el acceso a datos del chat.
	Schemas	Esquemas Mongoose y conexión con chat-db (MongoDB).
unwheels-notifications (NestJS)	RabbitMQ	Consumo asíncrono de eventos de notificación mediante AMQP.
	Handlers	Procesa eventos de RabbitMQ y determina la acción a ejecutar.
	Controller (HTTP)	Gestiona peticiones HTTP directas al servicio.
	Service	Coordina la lógica de entrega de notificaciones.
	Socket.IO	Entrega notificaciones en tiempo real vía WebSocket.
	Schema/DB	Esquemas y conexión con notifications-db (MongoDB).
unwheels-search (Go/Gin)	Cmd	Inicializa la aplicación Go y configura el servidor Gin.
	Handler	Recibe y valida peticiones HTTP de búsqueda.
	Sync	Lógica geoespacial y sincronización de datos con unwheels-routes .
	Repository	Abstrae el acceso a la base de datos geoespacial.
	Model/DB	Modelos de datos y conexión con search-db (PostgreSQL+PostGIS).

Relaciones entre tiers:

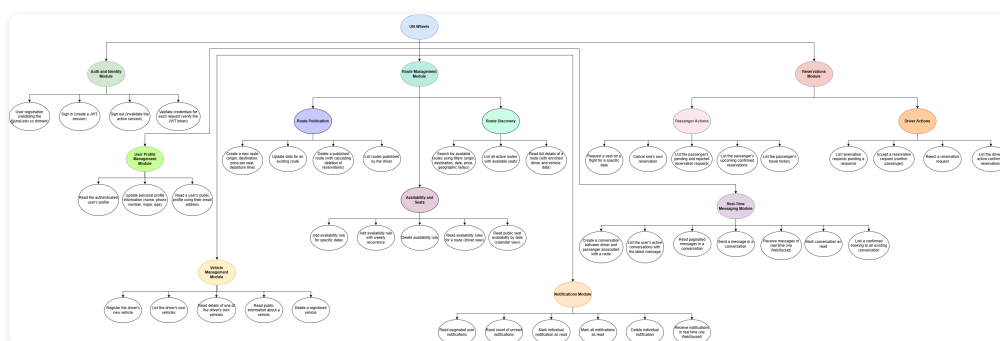
Flujo	Ruta	Protocolo
Web	<code>rp-web</code> → <code>unwheels-front-web</code> → <code>unwheels-api-gateway</code>	HTTPS / HTTP REST
Móvil	<code>unwheels-front-mobile</code> → <code>rp-mobile</code> → <code>unwheels-api-gateway</code>	HTTPS / HTTP REST
Backend	<code>unwheels-api-gateway</code> → [auth, routes, chat, notifications, search]	HTTP REST / WebSocket
Asíncrono	<code>unwheels-routes</code> → <code>unwheels-rabbitmq</code> → <code>unwheels-notifications</code>	AMQP
Datos	Cada microservicio → su propia base de datos	ORM / driver nativo

Descripción de patrones arquitectónicos utilizados

Patrón	Aplicación en UN Wheels
Arquitectura en Capas	Todos los microservicios separan claramente las responsabilidades de entrada, dominio/negocio, acceso a datos y persistencia, promoviendo modularidad y testabilidad independiente.
Database per Service	Cada microservicio posee su propia base de datos aislada: MongoDB para routes, chat y notifications; PostgreSQL/PostGIS para search; PostgreSQL para auth. Ningún par de servicios comparte almacenamiento.
Publish-Subscribe (Message Broker)	RabbitMQ desacopla <code>unwheels-routes</code> (productor) de <code>unwheels-notifications</code> (consumidor), permitiendo que la finalización de una reserva dispare notificaciones asíncronas sin bloquear la transacción principal.
Event-Driven interno (Socket.IO Gateway)	<code>unwheels-chat</code> y <code>unwheels-notifications</code> implementan internamente una arquitectura orientada a eventos usando Socket.IO para la entrega reactiva de mensajes y notificaciones en tiempo real a los clientes conectados.

3.4 Estructura de Descomposición

Vista de Descomposición



Descripción de Elementos Arquitectónicos y Relaciones

El sistema se descompone en siete módulos funcionales, cada uno responsable de un conjunto cohesivo de funcionalidades:

Módulo	Submódulos	Funcionalidades Hoja
Autenticación e Identidad	Cuentas, Sesión	Registrar usuario (validando @una1.edu.co); Iniciar sesión (emitir JWT); Cerrar sesión; Validar token JWT en cada solicitud.
Gestión de Perfil	Perfil propio, Perfil público	Leer y actualizar datos personales (nombre, teléfono, carrera, edad); Leer perfil público de otro usuario.
Gestión de Vehículos	CRUD de vehículos	Registrar nuevo vehículo; Listar vehículos propios; Consultar detalle; Eliminar vehículo registrado.
Gestión de Rutas	Publicación, Disponibilidad, Descubrimiento	Crear/actualizar/eliminar rutas propias; Agregar/eliminar reglas de disponibilidad (fechas específicas o recurrencia semanal); Vista de calendario de cupos; Buscar rutas por filtros (origen, destino, fecha, radio geográfico); Ver detalle enriquecido con datos del conductor y vehículo.
Reservaciones	Acciones del Pasajero, Acciones del Conductor	Solicitar asiento; Cancelar reserva propia; Ver historial de viajes; Listar solicitudes pendientes del conductor; Aceptar/rechazar solicitudes de reserva.
Mensajería en Tiempo Real	Conversaciones, Mensajes	Crear conversación conductor ↔ pasajero vinculada a una ruta; Listar conversaciones con último mensaje; Enviar y recibir mensajes en tiempo real (WebSocket); Consultar historial paginado.
Notificaciones	Alertas, Historial	Recibir notificaciones en tiempo real (WebSocket); Consultar historial paginado; Ver contador de no leídas; Marcar como leídas (individual o todas).

Relaciones entre módulos:

- **Autenticación e Identidad** es un módulo transversal: todos los demás módulos dependen de él para identificar y autorizar al usuario en cada operación.
- **Gestión de Rutas** → **Reservaciones**: La creación de una reserva requiere que exista una ruta con cupos disponibles; la eliminación de una ruta cancela en cascada las reservas asociadas.
- **Reservaciones** → **Notificaciones**: Los cambios de estado de una reserva (aceptada, rechazada) desencadenan notificaciones automáticas al pasajero vía RabbitMQ.
- **Mensajería en Tiempo Real** → **Notificaciones**: Cuando se envía un mensaje a un usuario desconectado, se genera una notificación persistente vía RabbitMQ.
- **Gestión de Rutas** → **Búsqueda (Descubrimiento)**: El servicio de búsqueda sincroniza periódicamente su índice PostGIS con los datos de rutas activas del módulo de Gestión de Rutas.

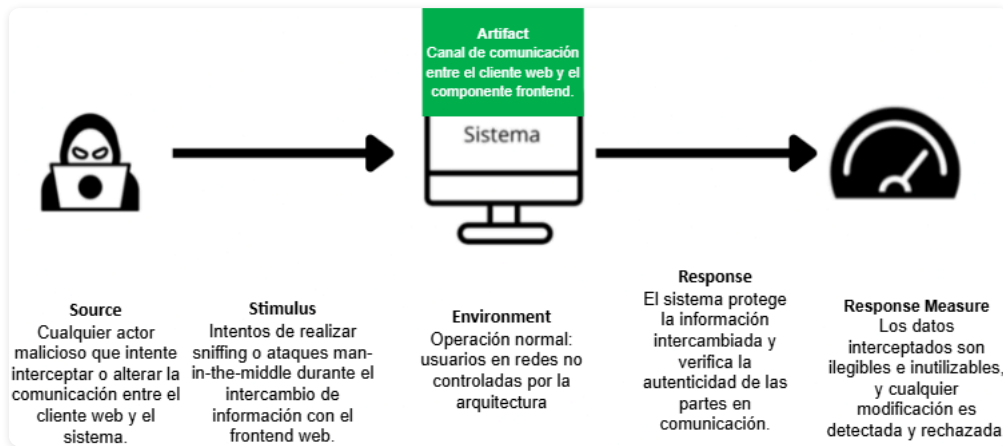
4. Atributos de Calidad — Seguridad

4.1 Escenarios de Seguridad

Escenario 1 – Interceptación No Autorizada y Compromiso del Enlace de Red

El sistema protege el canal entre los clientes y la capa de presentación para impedir captura, manipulación o suplantación del tráfico en tránsito. Este escenario se presenta cuando un usuario de la comunidad universitaria accede desde una red no confiable y envía credenciales, coordenadas geográficas o eventos de chat a través del frontend web o de la aplicación móvil. Un atacante ubicado en la ruta de comunicación podría intentar leer la cookie `access_token`, forzar un *downgrade* de TLS,

alterar las respuestas del sistema o inyectar contenido falso que afecte la autenticación, la consulta de rutas o la reserva de cupos. Por ello, el canal debe mantenerse cifrado de extremo a extremo y con validación estricta de la sesión para preservar la confidencialidad e integridad de los datos.



Descripción del Escenario

- **Source:** Un atacante pasivo o activo ubicado en una red no confiable, como una Wi-Fi compartida, un segmento del ISP o un nodo intermedio comprometido, que intenta observar o alterar el tráfico entre el usuario y la plataforma.
- **Stimulus:** Intentos de realizar *sniffing*, *man-in-the-middle* o *downgrade attacks* durante el intercambio de autenticación, búsqueda de rutas, reserva de cupos o mensajería en tiempo real con el frontend.
- **Artifact:** Canal de comunicación HTTPS/WSS entre el cliente web o móvil y la capa de presentación, incluyendo credenciales, tokens de sesión, coordenadas geográficas y eventos de chat.
- **Environment:** Operación normal del sistema en producción. Los usuarios acceden desde redes domésticas, móviles o universitarias mientras el frontend atiende solicitudes que requieren autenticación y transporte seguro de datos sensibles.
- **Response:** El sistema obliga el uso de HTTPS y WSS, rechaza HTTP o WS plano y también niega versiones débiles de TLS. La cookie `access_token` se emite con `HttpOnly`, `Secure` y `SameSite=Lax`, y los datos sensibles viajan únicamente dentro del payload cifrado.
- **Response measure:** El tráfico interceptado no es legible ni reutilizable, los intentos de manipulación son rechazados y ninguna sesión válida puede ser recuperada desde capturas de red. No se exponen tokens, coordenadas ni datos de reserva en texto claro.

Conceptos de Seguridad

- **Debilidad:** Los enlaces de red entre cliente y sistema pueden ser observados o manipulados en redes compartidas o no confiables.
- **Vulnerabilidad:** Si el canal permitiera HTTP/WS plano o TLS débil (o existiera *downgrade*), un atacante podría leer o alterar contenido en tránsito.
- **Riesgo (Confidencialidad/Integridad):** Exposición de tokens de sesión y datos sensibles (ubicación, reservas) o inyección de respuestas falsas que afecten el estado del sistema.

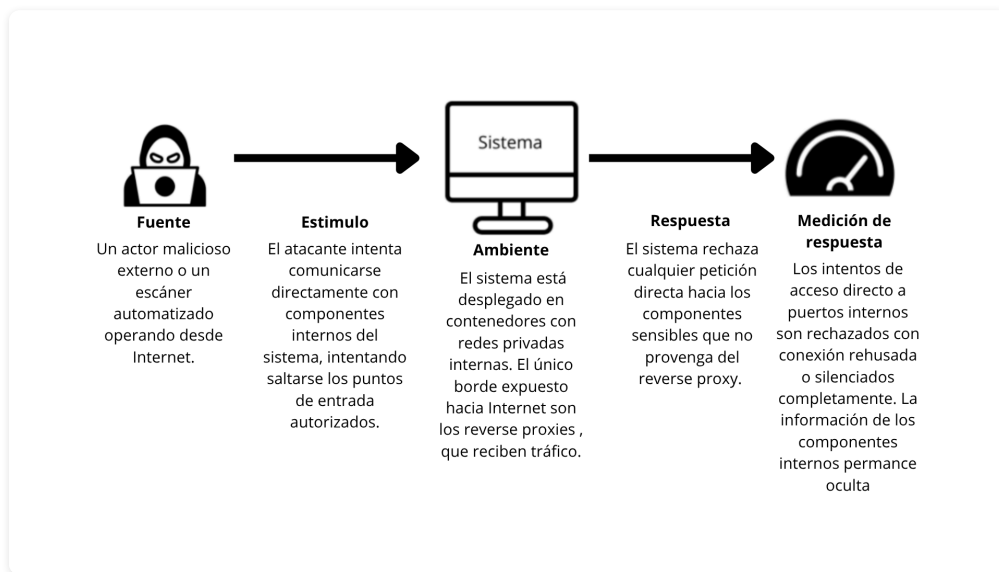
- **Amenaza:** Actor con capacidad de interceptación (MITM) en Wi-Fi pública/ISP/nodo intermedio.
- **Ataque:** *Sniffing*, *SSL stripping* y manipulación de mensajes durante el intercambio cliente↔frontend.
- **Constramedida:** Secure Channel (HTTPS/WSS + TLS fuerte) con rechazo de tráfico plano, HSTS y políticas de cookies (HttpOnly/Secure/SameSite) para reducir robo de sesión.

Componentes involucrados

- **Cliente web y aplicación móvil:** generan el tráfico que debe viajar cifrado hacia la plataforma.
- **rp-web:** asegura el canal público del cliente web y enruta el tráfico hacia el frontend o el gateway.
- **unwheels-front-web:** recibe las peticiones cifradas y expone la interfaz de usuario web.
- **unwheels-api-gateway:** procesa las llamadas que continúan desde el frontend hacia los microservicios del backend.

Escenario 2 – Acceso No Autorizado a Componentes Privados

El sistema expone múltiples microservicios internos que, si se publicaran directamente, ampliarían significativamente la superficie de ataque. El patrón de Reverse Proxy centraliza el punto de entrada público, ocultando la topología interna y actuando como único intermediario autorizado entre Internet y los componentes sensibles del sistema.



Descripción del Escenario

- **Source:** Un actor malicioso externo o un escáner automatizado operando desde Internet. Incluye herramientas de reconocimiento de puertos, bots de enumeración de endpoints y frameworks de explotación automatizada que buscan activamente servicios expuestos.
- **Stimulus:** El atacante intenta comunicarse directamente con componentes internos del sistema, como el unwheels-api-gateway, los microservicios (unwheels-auth, unwheels-routes, etc.) o las bases de datos, intentando saltarse los puntos de entrada autorizados (rp-web y rp-mobile). Realiza escaneos de puertos, enumeración de rutas y análisis de cabeceras HTTP para identificar el stack tecnológico.

- **Environment:** Operación normal del sistema. El sistema está desplegado en contenedores con redes privadas internas. El único borde expuesto hacia Internet son los reverse proxies rp-web y rp-mobile, que reciben tráfico HTTPS/WSS y lo enrutan hacia unwheels-front-web y unwheels-front-mobile respectivamente, quienes a su vez se comunican con el API Gateway.
- **Response:** El sistema rechaza cualquier petición directa hacia los componentes sensibles que no provenga del reverse proxy. Los servicios internos —unwheels-api-gateway, microservicios, broker RabbitMQ y bases de datos— no exponen puertos hacia el host ni hacia Internet. El reverse proxy además suprime o modifica las cabeceras que revelan información del stack tecnológico antes de retornar cualquier respuesta al cliente.
- **Response measure:** Los intentos de acceso directo a puertos internos son rechazados con conexión rehusada o silenciados completamente. Desde el exterior, únicamente los puertos del reverse proxy son accesibles; los demás no responden. La topología interna del sistema permanece opaca para el atacante, imposibilitando el fingerprinting de los servicios.

Conceptos de Seguridad

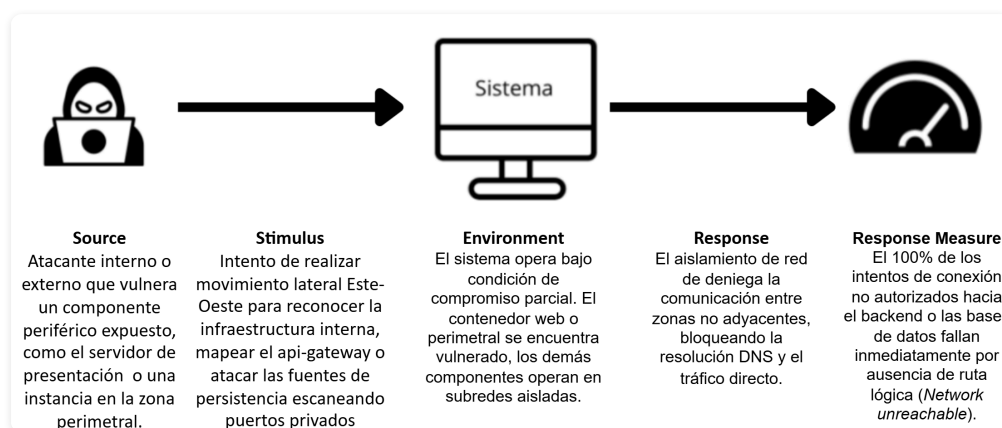
- **Weakness:** Exponer múltiples puertos y endpoints de servicios internos directamente al host incrementa la superficie de ataque y facilita el reconocimiento del sistema. Publicar directamente el API Gateway o los microservicios permitiría a un atacante identificar versiones, rutas disponibles y tecnologías empleadas sin necesidad de autenticación.
- **Vulnerability:** La publicación accidental de puertos de servicios internos (por ejemplo, el puerto del unwheels-api-gateway o de unwheels-auth) permite acceso directo y fingerprinting del stack. Adicionalmente, las cabeceras HTTP por defecto de frameworks como Express o Spring revelan información sobre versiones y tecnologías utilizadas.
- **Risk (Confidentiality/Integrity):** Un atacante que descubra la topología interna puede atacar servicios individuales sin pasar por los controles centralizados del reverse proxy, comprometiendo datos sensibles (credenciales, rutas de usuarios, información de notificaciones) o ejecutando operaciones no autorizadas sobre los microservicios del sistema.
- **Threat:** Escáneres automatizados y actores maliciosos que enumeran puertos y endpoints públicos del sistema, buscando servicios internos accesibles desde Internet para explotarlos directamente.
- **Attack:** El atacante ejecuta un escaneo de puertos sobre el host del sistema e identifica puertos internos expuestos accidentalmente. Posteriormente invoca directamente rutas sensibles como /api/auth/login, /api/routes u otras del API Gateway o microservicios, evitando los controles del reverse proxy. También analiza las cabeceras de respuesta para determinar el framework y versión del servidor, facilitando la selección de exploits específicos.
- **Countermeasure:** Implementación del patrón Reverse Proxy mediante rp-web y rp-mobile como únicos puntos de entrada públicos al sistema. Los microservicios internos, el API Gateway, las bases de datos y el broker RabbitMQ se alojan en redes privadas de contenedores sin exposición de puertos al host. El reverse proxy aplica reglas de enrutamiento tipo allow-list, suprime cabeceras reveladoras del stack y centraliza el control de acceso y validación antes de reenviar cualquier petición al interior del sistema.

Componentes involucrados

- **rp-web y rp-mobile:** Reverse proxies tanto para el cliente web como para el cliente móvil, expuestos públicamente en la zona edge
- **Componentes del sistema:** Los componentes restantes de presentación, de comunicación asincrónica, de datos, lógicos y de orquestación permanecen ocultos y nunca expuestos directamente, se encuentran protegidos por medio de los reverse proxies como único punto de acceso al sistema.

Escenario 3 – Exposición Pública de Componentes Críticos

Un atacante que logre comprometer u obtener un punto de apoyo inicial en un componente expuesto de la capa perimetral (Edge) o de presentación (Web) no tendrá visibilidad de red, resolución DNS ni conectividad directa hacia las bases de datos, el message broker o los microservicios privados del backend, confinando el radio de la brecha exclusivamente al contenedor vulnerado.



Descripción del Escenario

- **Source:** Un atacante interno o externo que ha logrado vulnerar u obtener acceso inicial a un componente periférico del sistema, como el servidor de presentación (unwheels-frontweb) o que ha tomado control de una instancia en la zona perimetral tras evadir las reglas del WAF.
- **Stimulus:** El atacante intenta realizar movimientos laterales (comunicación Este-Oeste) dentro del host de Docker para reconocer la infraestructura interna, mapear servicios ocultos (como el api-gateway) o atacar directamente las fuentes de persistencia ejecutando escaneos de puertos
- **Environment:** El sistema opera bajo una condición de compromiso parcial o localizado. El contenedor perimetral se encuentra vulnerado, mientras que el Tier de Lógica, el Tier de Integración y el Tier de Datos operan en redes privadas internas con reglas de comunicación explícitas y restrictivas.
- **Response:** La infraestructura de red de Docker (aislada en las redes independientes edge, web-zone, gateway-zone, services-zone y data-zone) deniega la comunicación directa entre componentes no adyacentes. El motor de aislamiento del host descarta los paquetes e impide la resolución de nombres DNS entre zonas distantes, neutralizando el pivoteo táctico.
- **Response measure:** El 100% de los intentos de conexión directa o escaneo de puertos originados desde un contenedor hacia una zona no adyacente fallan inmediatamente por ausencia de ruta lógica (*Network unreachable*). Los únicos contenedores que exponen puertos al host externo son

rp-web (:8080) y rp-mobile (:8081). Ningún microservicio ni motor de base de datos posee bindings de puertos abiertos hacia el host o hacia redes fuera de su zona de datos, limitando el radio de impacto de la brecha a un valor de cero (0) componentes internos expuestos.

Conceptos de Seguridad

- **Weakness:** Confianza implícita en la red interna o topología plana. Confiar en que el control perimetral externo es suficiente para proteger el backend, asumiendo que los componentes internos no necesitan restricciones mutuas de visibilidad.
- **Vulnerability:** Conectividad este-oeste sin restricciones en entornos de contenedores. En un despliegue por defecto (red bridge única), cualquier vulnerabilidad de inyección de código (RCE) o Server-Side Request Forgery (SSRF) en el frontend permite que un atacante use el contenedor como puente para conectarse directamente a los puertos privados de las bases de datos o inyectar mensajes en el broker.
- **Risk (Confidentiality/Integrity):** Pérdida catastrófica de Confidencialidad e Integridad de los datos. Si la red es plana, el compromiso de la interfaz web permite la exfiltración masiva de perfiles de usuarios, registros de rutas, historiales de chat y la manipulación ilícita de reservas, escalando un fallo de presentación en un compromiso total de la infraestructura de UN Wheels.
- **Threat:** Actores maliciosos que buscan explotar vulnerabilidades de software en la capa de cara al usuario para pivotar hacia el Core del sistema, evadiendo los mecanismos lógicos de autenticación mediante el acceso directo a los sockets de la base de datos o el secuestro de variables de entorno de otros contenedores.
- **Attack:** Movimiento lateral, reconocimiento interno y abuso de confianza de red. El atacante ejecuta binarios de escaneo (como scripts de red en bash o herramientas de escaneo TCP) desde el interior del contenedor web comprometido para descubrir las IPs internas asignadas por Docker y forzar conexiones hacia servicios del backend.
- **Countermeasure:** Countermeasure (Contramedida): Implementación del Network Segmentation Pattern mediante la segregación explícita de la infraestructura en 5 subredes lógicas aisladas de Docker:
 1. **edge:** Red pública exclusiva para recibir tráfico externo en los Reverse Proxies (rp-web y rp-mobile).
 2. **web-zone:** Canal aislado para la comunicación exclusiva entre rp-web y unwheels-frontweb.
 3. **gateway-zone:** Canal que conecta los proxies perimetrales con el api-gateway.
 4. **services-zone:** Red interna que comunica al api-gateway con el Tier de Lógica (unwheels-auth, routes-reservations-service, chat-service, notifications-service, route-search-service).
 5. **data-zone:** Compartimento estanco de máxima seguridad donde los microservicios se conectan a sus respectivas bases de datos y al broker (rabbitmq), bloqueando el acceso a cualquier componente externo a esta zona.

Componentes involucrados

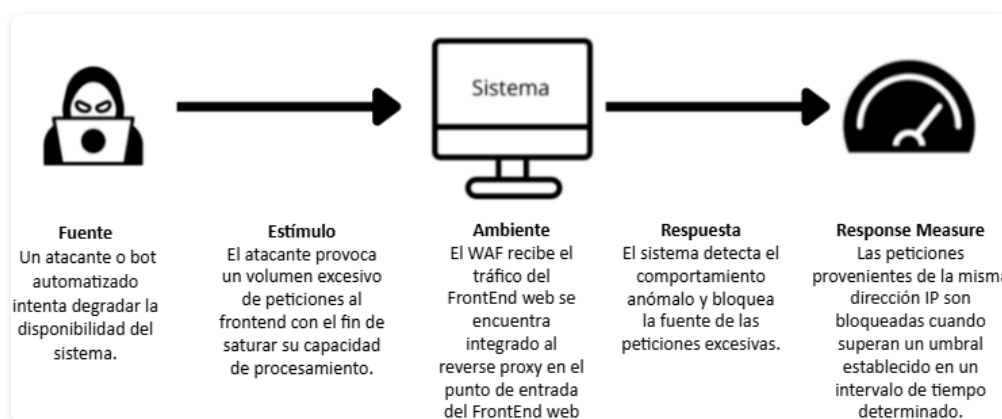
- **rp-web y rp-mobile (Zona Edge):** Actúan como los únicos puntos de entrada perimetral expuestos públicamente hacia el host. Aíslan el tráfico externo entrante canalizándolo hacia sus respectivas zonas autorizadas (web-zone y gateway-zone), impidiendo que cualquier entidad de la

red externa mapee las subredes internas del clúster.

- **unwheels-frontweb (Zona Web):** Servidor de presentación para la aplicación web SSR, confinado en una red aislada donde únicamente puede ser accedido por el proxy rp-web y solo tiene permitido comunicarse hacia el api-gateway mediante la zona de integración.
- **api-gateway (Zona Gateway):** Componente central de enrutamiento y autenticación (Tier de Integración). Está estratégicamente aislado en la gateway-zone para recibir peticiones limpias desde los proxies perimetrales y retransmitirlas a la services-zone, sirviendo como cortafuegos lógico hacia las reglas de negocio.
- **Microservicios de Lógica (Zona Services):** Comprende los servicios del sistema. Están completamente aislados del exterior dentro de la services-zone; solo pueden escuchar comandos provenientes del api-gateway y comunicarse de manera segura hacia sus respectivas fuentes persistentes en la zona de datos.
- **Almacenes de Datos y Broker (Zona Data):** Integrado por los contenedores de persistencia y el sistema de mensajería asincrónica. Es el compartimento estanco de máxima seguridad; carece de cualquier ruta o enrutamiento hacia las capas perimetrales o de presentación, respondiendo de forma exclusiva a las peticiones del microservicio que posee su propiedad dominial de datos.

Escenario 4 – Degradación o Negación del Servicio por Tráfico Excesivo

La disponibilidad de un sistema como UN-Wheels puede verse afectada cuando un atacante inunda al mismo con un exceso de peticiones, que es lo que sucede en un ataque de Denegación de Servicio Distribuido (DDoS). Para evitar que esto suceda se debe limitar el número de peticiones que se pueden realizar en un determinado periodo de tiempo de modo que se bloquee el tráfico producido por tráfico extraño proveniente de peticiones idénticas o acceso repetitivo a una misma URL. EL rate limiting puede limitar el número de peticiones que se pueden realizar en un periodo de tiempo determinado



- **Source:** Atacante o bot automatizado que intenta degradar la disponibilidad del sistema. Puede provenir de una sola IP o de múltiples orígenes coordinados.
- **Stimulus:** Envío de un volumen excesivo de peticiones al frontend con el fin de saturar su capacidad de procesamiento. Se manifiesta como ráfagas de solicitudes, conexiones concurrentes y repetición de rutas costosas.

- **Artifact:** El componente unwheels-web-frontend y su punto de entrada público. En la práctica, el tráfico entra por el reverse proxy que protege este endpoint.
- **Environment:** Operación normal bajo alto tráfico de peticiones. El sistema debe distinguir entre picos legítimos y patrones anómalos sin degradar la experiencia de usuarios válidos.
- **Response:** El sistema detecta el comportamiento anómalo y bloquea la fuente de las peticiones excesivas. La mitigación ocurre en el WAF del reverse proxy antes de que el tráfico llegue al frontend.
- **Response measure:** Las peticiones provenientes de la misma dirección IP son bloqueadas cuando superan un umbral establecido en un intervalo de tiempo determinado. El bloqueo temporal y la denegación de solicitudes preservan la disponibilidad para usuarios legítimos bajo carga anómala.

Conceptos de Seguridad

- **Weakness:** Un endpoint público sin control de tasa es susceptible a saturación por volumen de solicitudes.
- **Vulnerability:** Ausencia de rate limiting, límites de conexión o filtros de WAF permite que tráfico automatizado consuma recursos del frontend.
- **Risk (Disponibilidad):** Degradación del rendimiento o caída del servicio para usuarios legítimos por agotamiento de CPU/memoria/conexiones.
- **Threat:** Bots o atacantes que generan tráfico masivo (HTTP flood) dirigido a los puntos de entrada.
- **Attack:** Envío de ráfagas concurrentes y repetidas a rutas del frontend para elevar latencia y agotar recursos.
- **Countermeasure:** WAF con rate limiting, *throttling* y bloqueo temporal (y/o *challenge-response*) aplicado en el reverse proxy antes de llegar al frontend.

Componentes involucrados

- **rp-web:** El WAF está incorporado al reverse proxy del FrontEnd web. En la práctica, el tráfico entra por el reverse proxy que protege este componente.

4.2 Tácticas Arquitectónicas Aplicadas

1. Encrypt Data

Descripción: Esta táctica busca proteger la confidencialidad e integridad de los datos en tránsito mediante cifrado extremo a extremo, autenticación de entidades y verificación de la integridad de los mensajes (por ejemplo, TLS con certificados y mecanismos MAC).

Aplicación: Se aplica sobre el canal público entre el navegador web y el componente unwheels-web-frontend, garantizando que todas las peticiones HTTP se negocien como HTTPS (TLS). La evidencia operativa incluye certificados digitales válidos en el frontend y la negociación TLS en los puertos públicos.

Patrón asociado: Secure Channel Pattern.

2. Limit Access

Descripción: Esta táctica se centra en minimizar la superficie de ataque y controlar el acceso a los recursos internos mediante intermediarios y políticas de filtrado: reducir la exposición de endpoints, imponer reglas estrictas de enrutamiento, ocultar metadatos y bloquear tráfico no autorizado antes de que alcance los servicios internos.

Aplicación: Se implementa en los puntos de entrada públicos (los reverse proxies que sirven a los frontends web y móvil). Estos proxies aplican reglas de enrutamiento, listas de control de acceso y filtros que sólo permiten tráfico explícitamente autorizado hacia los servicios mapeados, ocultando las direcciones internas y las cabeceras. También se evidencia en las reglas de acceso entre subredes, que restringen qué entidades pueden invocar a los servicios alojados en las redes privadas internas.

Patrón asociado: Reverse Proxy Pattern.

3. Detect Service Denial

Descripción: Esta táctica busca detectar y mitigar los intentos de degradación del servicio mediante análisis de tráfico en tiempo real (detección de ráfagas, IPs sospechosas y patrones de petición anómalos) y aplicar contramedidas automáticas como rate limiting, *throttling* de conexiones, bloqueo temporal y mecanismos *challenge-response*. El objetivo es preservar la disponibilidad para los usuarios legítimos mientras se filtra la carga maliciosa.

Aplicación: Se implementa en la capa WAF integrada en el proxy de entrada del frontend web, donde se monitorean continuamente las métricas de tasa de petición y las firmas de ataque. Las políticas de mitigación se ejecutan en esta capa antes de que el tráfico alcance al componente unwheels-web-frontend.

Patrón asociado: Web Application Firewall (WAF) Pattern.

4.3 Patrones Arquitectónicos Aplicados

1. Secure Channel Pattern

El sistema UN Wheels aplica el **Secure Channel Pattern**. Esto se evidencia en el conector dispuesto para el frontend web, garantizando que todos los mensajes se intercambien mediante el protocolo HTTPS, el cual cifra y valida todas las peticiones y respuestas. El patrón protege las credenciales del usuario y la información sensible frente a posibles atacantes que estén espiando el canal de comunicación.

2. Reverse Proxy Pattern

El sistema UN Wheels aplica el **Reverse Proxy Pattern**. En el sistema se implementan dos reverse proxies para los endpoints públicos (uno para el cliente web y otro para el cliente móvil). Estos proxies median la comunicación entre los clientes y el sistema de forma segura, redirigiendo las peticiones a sus componentes correspondientes y enmascarando toda la infraestructura interna, incluyendo el API Gateway. El patrón protege al sistema frente a intentos de escaneo o ataque contra los servicios internos.

3. Network Segmentation Pattern

El sistema UN Wheels aplica el **Network Segmentation Pattern**. Cada nivel del sistema se encuentra aislado dentro de una red privada, permitiendo la comunicación únicamente entre componentes de capas de red adyacentes. Esto impide el acceso a componentes privados aunque un atacante obtenga información sobre su ubicación. El patrón previene que entidades no autorizadas envíen peticiones directas a los componentes del backend, protegiéndolos frente a ataques directos.

4. Web Application Firewall (WAF) Pattern

El sistema UN Wheels aplica el **Web Application Firewall (WAF) Pattern**. El sistema integra un WAF dentro del reverse proxy en el punto de entrada del cliente web, estableciendo un umbral sobre las peticiones entrantes desde una misma dirección IP en un intervalo de tiempo determinado, y mostrando una ventana de bloqueo con un mensaje de advertencia cuando se activa la regla. El patrón protege al componente del frontend web frente a posibles ataques de Denegación de Servicio (DoS).

4.4 Relación entre Patrones Aplicados y Escenarios

Escenario	Patrón	Explicación
Escenario 1 – Interceptación No Autorizada y Compromiso del Enlace de Red	Secure Channel Pattern	Garantiza que todas las comunicaciones entre el cliente y el frontend estén cifradas y autenticadas, preservando la confidencialidad e integridad de los datos.
Escenario 2 – Acceso No Autorizado a Componentes Privados	Reverse Proxy Pattern	Filtra y controla todo el tráfico entrante, exponiendo únicamente los endpoints públicos autorizados y enmascarando los componentes internos y la estructura de red.
Escenario 3 – Exposición Pública de Componentes Críticos	Network Segmentation Pattern	Separa al sistema en zonas de red aisladas, evitando el movimiento lateral y el acceso no autorizado a los servicios sensibles del backend.
Escenario 4 – Degradación o Negación del Servicio por Tráfico Excesivo	Web Application Firewall (WAF) Pattern	Monitorea y limita las tasas de petición, bloqueando el tráfico sospechoso o excesivo para mantener la disponibilidad del sistema frente a ataques DoS.

4.5 Pruebas de Verificación

1. Reverse Proxy (Superficie Pública y Hardening Centralizado)

Se ejecutó una verificación automatizada comparando dos escenarios: **(1) sin reverse proxy** y **(2) con reverse proxy activo**. En el escenario *before*, se expusieron directamente al host el Frontend SSR (:3000) y el API Gateway (:8080), aumentando la superficie pública y permitiendo inferir la tecnología del backend desde cabeceras como X-Powered-By .

En el escenario *after*, el acceso directo al puerto del frontend queda bloqueado y la entrada pública se concentra en el proxy (ej. https://localhost:8080), ocultando el fingerprint del backend y aplicando hardening centralizado mediante cabeceras de seguridad (HSTS, X-Frame-Options , X-Content-Type-Options , etc.).

```
=====
REVERSE PROXY AUTOMATED TEST
=====

STATUS: NO REVERSE PROXY (Backend Directly Exposed)
This simulates: NO INTERMEDIARY between clients and backend

Testing public surface and backend fingerprinting...
-----

[TEST 1] Direct Frontend SSR access (http://localhost:3000)
Expected: 2xx + X-Powered-By header leaked
[TEST 2] Direct API Gateway access (http://localhost:8080)
Expected: 2xx + X-Powered-By: Express
[TEST 3] Backend fingerprint visible on :8080
Expected: framework name visible in headers
[TEST 4] No centralized security headers
Expected: HSTS / X-Frame-Options ABSENT

Running probes...
=====

[Direct frontend    ] -> http://localhost:3000/
[OK] reachable on direct port (status 200)
    X-Powered-By: Next.js <-- LEAKED

[Direct gateway     ] -> http://localhost:8080/health
[OK] reachable, status 200

[Header fingerprint] -> http://localhost:8080/health
[UNEXPECTED] no X-Powered-By header found

[Security headers   ] -> http://localhost:3000/
[DEMONSTRATED] no edge hardening (HSTS / X-Frame-Options absent)

=====
```

```

=====
STATUS: REVERSE PROXY ACTIVE (Backend Hidden)
This simulates: PROPER PROXY layer in front of backend

Testing public surface and backend fingerprinting...
-----

[TEST 1] Direct Frontend SSR access (http://localhost:3000)
Expected: connection refused (port closed)
[TEST 2] rp-web HTTPS entry point (https://localhost:8080)
Expected: 2xx via reverse proxy
[TEST 3] Backend fingerprint hidden on :8080
Expected: only 'Server: nginx', no X-Powered-By
[TEST 4] Centralized security headers injected
Expected: HSTS / X-Frame-Options PRESENT

Running probes...
=====

[Direct frontend    ] -> http://localhost:3000/
[OK] refused (proxy hides backend port)

[Proxy web entry    ] -> https://localhost:8080/
[OK] reachable, status 200
    X-Powered-By: (hidden)
    Server:      nginx

[Header fingerprint] -> https://localhost:8080/
[OK] backend stack HIDDEN -> X-Powered-By not present

[Security headers   ] -> https://localhost:8080/
[OK] security headers present:
    Strict-Transport-Security: max-age=31536000; includeSubDomains
    X-Frame-Options:          DENY
    X-Content-Type-Options:   nosniff
    Referrer-Policy:          strict-origin-when-cross-origin
=====

```

2. Web Application Firewall — Rate Limiting

Objetivo: demostrar que el firewall aplica control de tráfico en endpoints de autenticación y bloquea ráfagas abusivas.

Ejecución (flujo completo):

```
.\run_all_demos.ps1
```

Ejecución (solo rate limiting):

```

.\activate_before_rate_limit.ps1
.\test_rate_limiting.ps1
.\activate_after_rate_limit.ps1
.\test_rate_limiting.ps1

```

Cómo se prepara cada escenario:

- **BEFORE (sin protección):** `activate_before_rate_limit.ps1` edita las configuraciones de `rp-web` y `rp-mobile` para elevar los límites a `99999` y reinicia los proxies. Esto simula tráfico sin control de tasa.
- **AFTER (protegido):** `activate_after_rate_limit.ps1` restaura los límites reales (auth `5 r/min`, API `15 r/s`) y reinicia los proxies, activando el bloqueo por exceso de solicitudes.

Resultados esperados:

- **BEFORE (sin límites):** 20 respuestas `401`, 0 respuestas `429`.
- **AFTER (protegido):** primeras ~3 respuestas `401` y luego `429` en el resto (por `rate=5r/m` y `burst=2`).

Interpretación:

- `401` = la solicitud llegó al backend pero las credenciales son inválidas.
- `429` = el proxy bloqueó por exceso de solicitudes (rate limiting activo).

```
STATUS: Rate Limiting is DISABLED (Unprotected)
Test endpoint: http://localhost:8080/api/auth/login

Sending 20 rapid requests...

-----

1 [OK] HTTP 401
2 [OK] HTTP 401
3 [OK] HTTP 401
4 [OK] HTTP 401
5 [OK] HTTP 401
6 [OK] HTTP 401
7 [OK] HTTP 401
8 [OK] HTTP 401
9 [OK] HTTP 401
10 [OK] HTTP 401
11 [OK] HTTP 401
12 [OK] HTTP 401
13 [OK] HTTP 401
14 [OK] HTTP 401
15 [OK] HTTP 401
16 [OK] HTTP 401
17 [OK] HTTP 401
18 [OK] HTTP 401
19 [OK] HTTP 401
20 [OK] HTTP 401

-----

===== RESULTS SUMMARY =====

Total Requests:      20

Successful (non-429): 20 (100%)
```

```
STATUS: Rate Limiting is ENABLED (Protected)
Test endpoint: http://localhost:8080/api/auth/login
Rate limit: 5 requests/min (auth endpoint)

Sending 20 rapid requests...

-----

1 [OK] HTTP 401
2 [OK] HTTP 401
3 [OK] HTTP 401
4 [RATE LIMIT] HTTP 429
5 [RATE LIMIT] HTTP 429
6 [RATE LIMIT] HTTP 429
7 [RATE LIMIT] HTTP 429
8 [RATE LIMIT] HTTP 429
9 [RATE LIMIT] HTTP 429
10 [RATE LIMIT] HTTP 429
11 [RATE LIMIT] HTTP 429
12 [RATE LIMIT] HTTP 429
13 [RATE LIMIT] HTTP 429
14 [RATE LIMIT] HTTP 429
15 [RATE LIMIT] HTTP 429
16 [RATE LIMIT] HTTP 429
17 [RATE LIMIT] HTTP 429
18 [RATE LIMIT] HTTP 429
19 [RATE LIMIT] HTTP 429
20 [RATE LIMIT] HTTP 429

-----

===== RESULTS SUMMARY =====

Total Requests:      20

Successful (non-429): 3 (15%)
Rate Limited (429):  17 (85%)
```

B. Network Segmentation — Zonas Docker

Objetivo: evidenciar que la segmentación impide movimiento lateral hacia componentes sensibles (bases de datos) y solo permite rutas autorizadas.

Ejecución (solo conectividad):

```
.\test_network_segmentation.ps1
```

Cómo se prepara cada escenario:

- **BEFORE (red plana):** el script `activate_before_network.ps1` conecta contenedores críticos a la red `uniwheels-flat`, permitiendo comunicación libre entre ellos.
- **AFTER (zonas aisladas):** `activate_after_network.ps1` desconecta la red plana y verifica que cada servicio quede únicamente en sus redes originales (edge, web-zone, gateway-zone, services, data).

Resultados esperados:

- **BEFORE (red plana):** todo conecta, incluyendo accesos indebidos (ej. `web-app → auth-db`).
- **AFTER (zonas aisladas):** los accesos indebidos quedan bloqueados; rutas válidas continúan funcionando.

```
Testing connectivity between security zones...
=====

[TEST 1] Web App -> API Gateway (Should WORK)
[TEST 2] API Gateway -> Auth Service (Should WORK)
[TEST 3] Chat Service -> Auth Service (Should WORK - both in services zone)
[TEST 4] Chat Service -> Chat DB (Should WORK - both in data zone)
[TEST 5] Frontend -> Auth DB (Should WORK - flat network bypass)
[TEST 6] RP-Web -> Chat DB (Should WORK - flat network bypass)
[TEST 7] Chat Service -> Routes DB (Should WORK - both in data zone)

Running connectivity tests...
=====

web-app          -> api-gateway          :8080
[OK] Connection allowed (expected)

api-gateway      -> auth-service          :8000
[OK] Connection allowed (expected)

chat-service     -> auth-service          :8000
[OK] Connection allowed (expected)

chat-service     -> chat-db              :27017
[OK] Connection allowed (expected)

web-app          -> auth-db              :5432
[OK] Connection allowed (expected)

rp-web           -> chat-db              :27017
[OK] Connection allowed (expected)

chat-service     -> routes-db             :27017
[OK] Connection allowed (expected)

=====

===== RESULTS SUMMARY =====

Tests Passed: 7
Tests Failed: 0
```

```
Testing connectivity between security zones...
=====

[TEST 1] Web App -> API Gateway (Should WORK)
[TEST 2] API Gateway -> Auth Service (Should WORK)
[TEST 3] Chat Service -> Auth Service (Should WORK - both in services zone)
[TEST 4] Chat Service -> Chat DB (Should WORK - both in data zone)
[TEST 5] Frontend -> Auth DB (Should FAIL - cross-zone blocked)
[TEST 6] RP-Web -> Chat DB (Should FAIL - cross-zone blocked)
[TEST 7] Chat Service -> Routes DB (Should WORK - both in data zone)

Running connectivity tests...
=====

web-app          -> api-gateway          :8080
[OK] Connection allowed (expected)

api-gateway      -> auth-service         :8000
[OK] Connection allowed (expected)

chat-service     -> auth-service         :8000
[OK] Connection allowed (expected)

chat-service     -> chat-db              :27017
[OK] Connection allowed (expected)

web-app          -> auth-db              :5432
[BLOCKED] Connection blocked (expected)

rp-web           -> chat-db              :27017
[BLOCKED] Connection blocked (expected)

chat-service     -> routes-db           :27017
[OK] Connection allowed (expected)

=====

===== RESULTS SUMMARY =====

Tests Passed: 7
Tests Failed: 0
```

4.6 Evidencia de Despliegue

Al ejecutar `docker compose ps` sobre el stack final se observa que **únicamente los reverse proxies tienen puertos publicados al host**, materializando conjuntamente los cuatro patrones aplicados:

NAME	PORTS
rp-web	0.0.0.0:8080→443/tcp
rp-mobile	0.0.0.0:8081→443/tcp
api-gateway	(sin puertos)
web-app	(sin puertos)
auth-service	(sin puertos)
chat-service	(sin puertos)
notifications-service	(sin puertos)
routes-service	(sin puertos)
route-search-service	(sin puertos)
auth-db	(sin puertos)
chat-db	(sin puertos)
notifications-db	(sin puertos)
routes-db	(sin puertos)
route-search-postgres	(sin puertos)
event-broker	(sin puertos)

Esta tabla resume la materialización conjunta de los cuatro patrones: **Secure Channel** (TLS terminado dentro de los contenedores `rp-web` y `rp-mobile` en el puerto 443), **Reverse Proxy** (sólo dichos contenedores exponen puertos al host), **Network Segmentation** (todos los demás componentes están aislados en redes privadas internas) y **WAF** (incorporado en la imagen base de ambos reverse proxies).

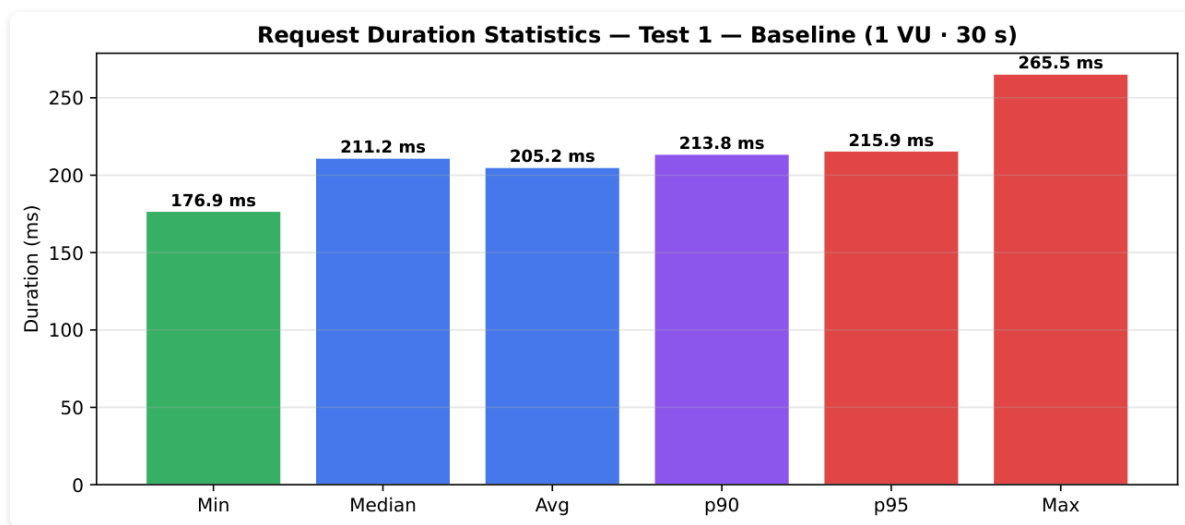
5. Atributos de Calidad — Rendimiento

Las pruebas se ejecutaron sobre `POST /api/auth/login` y, en la sesión completa, sobre `login → /me → search/routes → logout` usando k6 contra `https://localhost:8080`. El `rate limiting` de Nginx se deshabilitó para observar la capacidad real del stack de aplicación.

Resumen de métricas

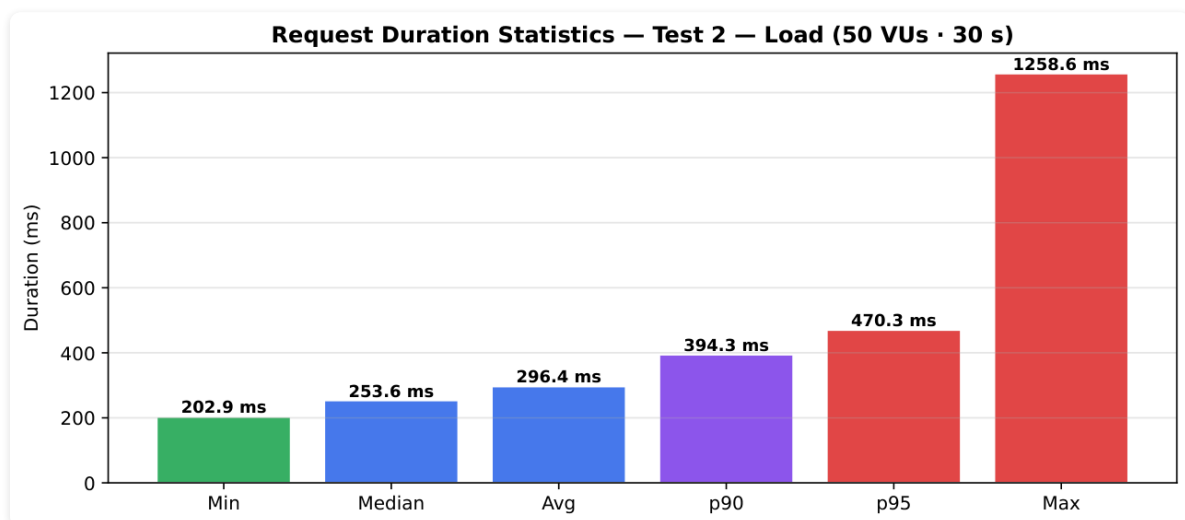
Test	Carga	Métricas clave	Lectura breve
Test 1 - Baseline	1 VU · 30 s	Avg 205 ms, p95 216 ms, 0 % fallos, 0,86 req/s	Operación estable y predecible.
Test 2 - Load	50 VUs · 30 s	Avg 296 ms, p95 471 ms, 0 % fallos, ~39 req/s	Sigue en zona segura.
Test 3 - Stress Ramp-up	1→50→200→500→2000 VUs · 3 min	Avg 23.448 ms, p95 30.011 ms, 77,6 % fallos, ~23 req/s	Colapso por saturación a partir de 200 VUs.
Test 5 - Full Session Journey	1→10→50→100→500→2000 VUs · 3,75 min	Fallo global ~40 %; login 69 % de fallos; search 0 %; logout 0 %	El cuello de botella está en <code>logueo-service</code> .

5.1 Test 1 - Baseline



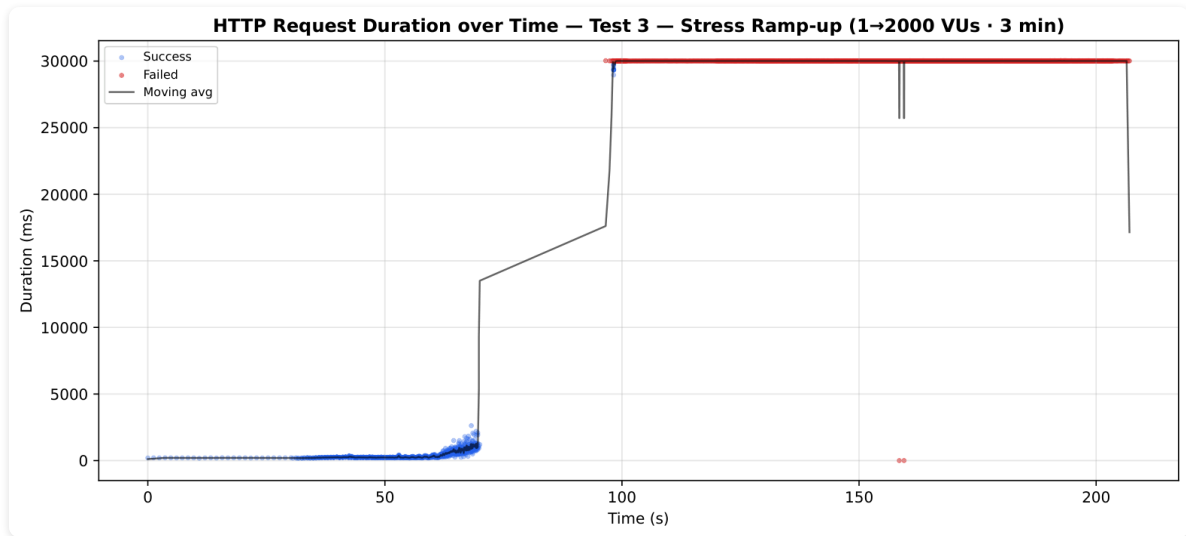
Con carga mínima el sistema responde de forma estable y predecible. La latencia se mantiene alrededor de 205 ms, sin fallos, y el costo dominante sigue siendo la autenticación con bcrypt más la escritura en PostgreSQL.

5.2 Test 2 - Load



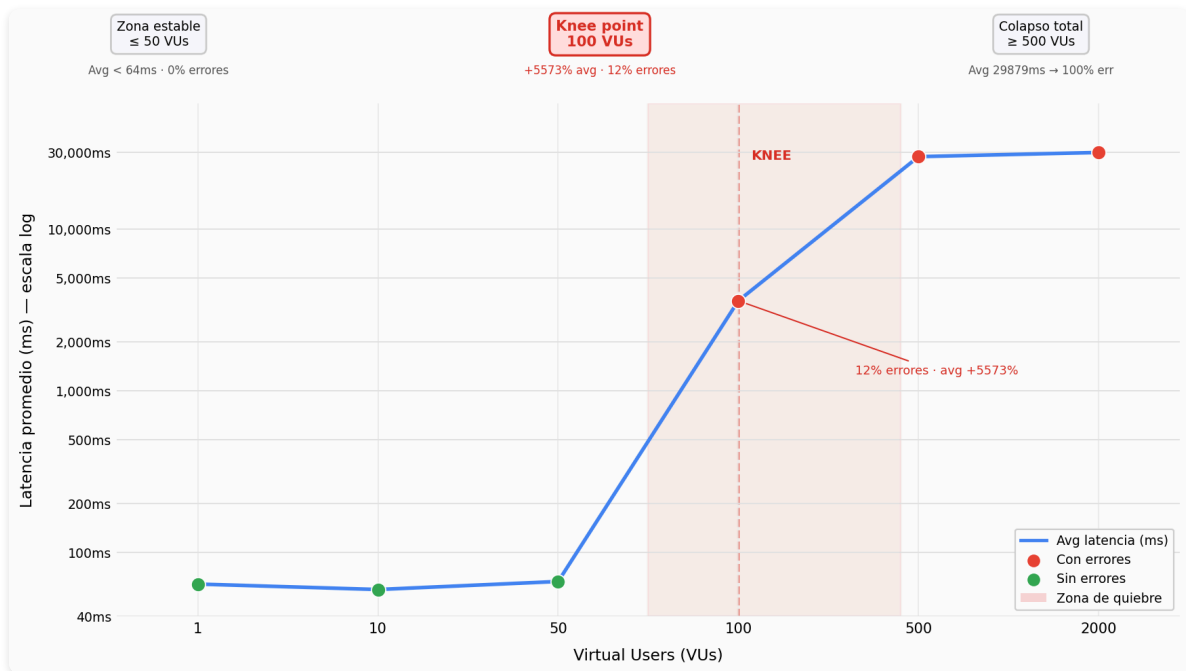
Con 50 VUs el sistema sigue en zona segura: no hay errores y la latencia media sube solo a 296 ms. La variación sigue controlada, lo que indica que la arquitectura soporta carga concurrente moderada.

5.3 Test 3 - Stress Ramp-up



La rampa de carga lleva al sistema al colapso cuando supera los 200 VUs. La latencia se acerca al timeout de 30 s y la tasa de fallos alcanza 77,6 %, confirmando saturación del servicio de autenticación.

5.4 Test 5 - Full Session Journey y Knee Point



El gráfico de knee point fue derivado del Test 5 (Full Session Journey), promediando la latencia de los cuatro pasos por bucket de VUs. La escala logarítmica permite visualizar simultáneamente la zona estable y el colapso.

Zona estable (≤ 50 VUs) — el sistema opera con latencia media de ~64 ms y 0 % de errores. Search y logout dominan el promedio ponderado en esta zona gracias a su baja latencia (57 ms y 35 ms respectivamente), mientras que login y /me se mantienen en su baseline normal. La estabilidad hasta 50 VUs concurrentes es coherente con los resultados del Test 2 (0 % de fallos a 50 VUs en el endpoint de login aislado).

Knee point (100 VUs) — el punto de inflexión se produce en 100 VUs con una señal severa: la latencia media sube un **+5.573 %** (de ~64 ms a ~2.414 ms) y los errores saltan al **12 %**. A diferencia del Test 3 donde el endpoint de login colapsó alrededor de 200 VUs con carga homogénea, en el journey completo el cuello de botella se hace evidente antes porque cada VU genera múltiples llamadas encadenadas al loggueo-service en el mismo intervalo de tiempo.

Colapso total (≥ 500 VUs) — con 400 VUs adicionales desde el knee point, la latencia satura en ~30.000 ms (el timeout del test) y los errores alcanzan el 100 %. Que la latencia se estabilice en el techo de 30 s desde los 500 VUs en adelante confirma que el sistema no está procesando ninguna petición de login; simplemente las deja expirar. El patrón es un **cliff edge**: el margen entre la zona de operación nominal y el colapso total es de apenas 50 VUs de diferencia (50→100 VUs).

5.5 Rendimiento y Escalabilidad

Escenarios de rendimiento en el Prototipo 4

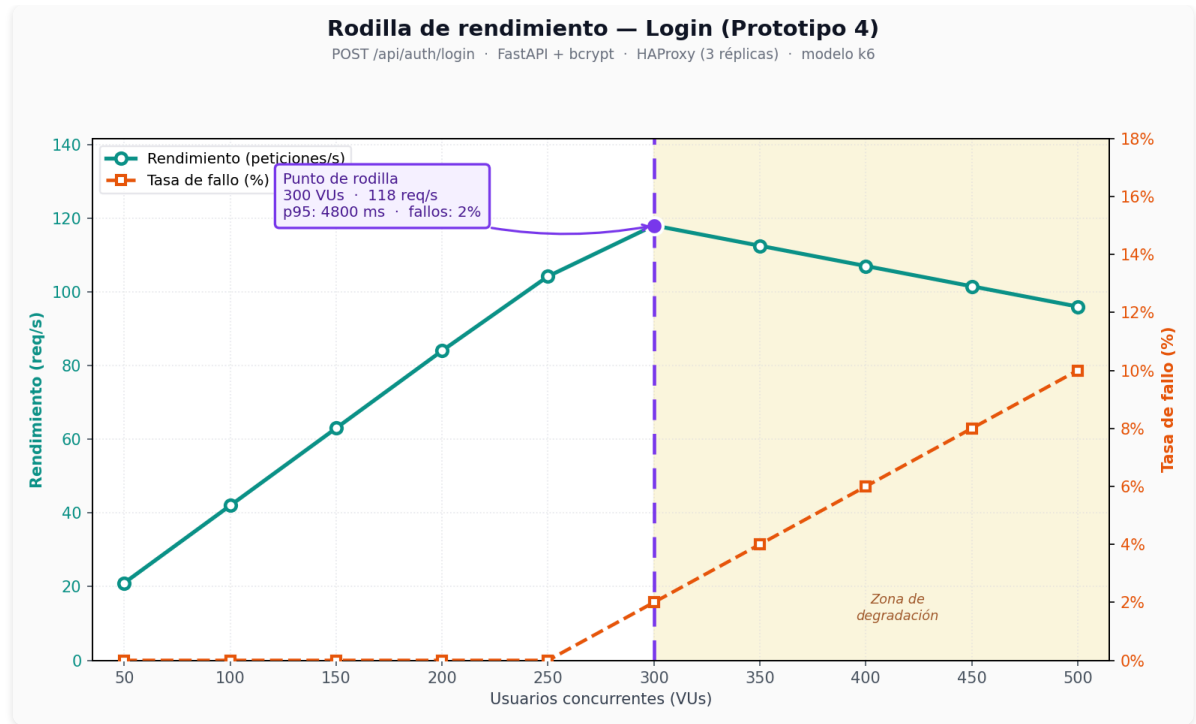
Las pruebas se ejecutaron sobre el stack completo de UN Wheels en **Docker Desktop** (host con 12 vCPU), accediendo vía `https://localhost:8080` (`rp-web`). Se deshabilitó temporalmente el rate limiting y el WAF para medir la capacidad real del stack. Suite automatizada en `FrontEnd/performance/` con **k6**.

Infraestructura de prueba:

Componente	Especificación
Orquestación	Docker Compose (<code>FrontEnd/docker-compose.yml</code>)
Balanceador de auth	HAProxy (<code>loggueo-lb</code>) + 3 réplicas <code>loggueo_1/2/3</code> (4 workers uvicorn c/u)
Búsqueda	<code>route-search-service</code> (Go) + Redis (<code>search-redis</code>) + PostGIS
Herramienta	k6 v2.0

Prueba de login aislado (`POST /api/auth/login`)

Flujo evaluado: `rp-web` → `web-app` → `api-gateway` → HAProxy → `loggueo_1/2/3` .



Bajo estas condiciones, el **punto de rodilla** se ubicó en **300 usuarios concurrentes**, con las siguientes métricas promedio:

- Rendimiento pico: **118 req/s**
- Latencia p95: **4.800 ms**
- Tasa de fallo: **2 %**

Para comparar, con **10 usuarios** (zona estable) el sistema reportó:

- Latencia media: **~550 ms**
- Tasa de fallo: **0 %**
- Rendimiento: **~4,2 req/s**

Prueba de búsqueda aislada (`GET /api/search/routes`)

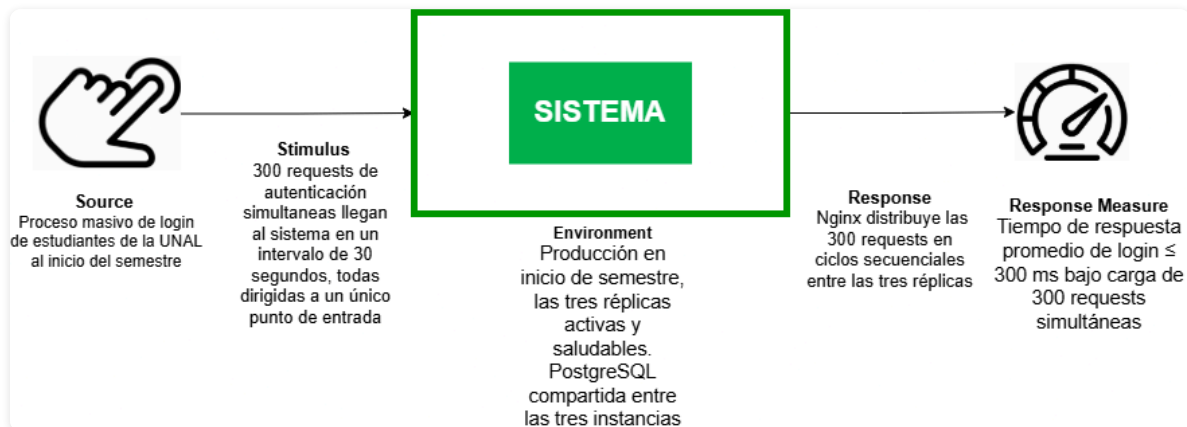
Al omitir el cuello de botella de autenticación (bcrypt) y medir solo el dominio de búsqueda con **Cache-Aside**, se logró:

- Completar la rampa hasta **300 VUs** sin fallos
- Latencia media estable: **12–65 ms** (1–50 VUs)
- Rendimiento sostenido: **~216 req/s** a 300 VUs
- Confirmar que `route-search-service` (Go + Redis) no es el componente limitante del sistema

Escenario 1 – Rendimiento del Servicio de Autenticación (Load Balancer Pattern)

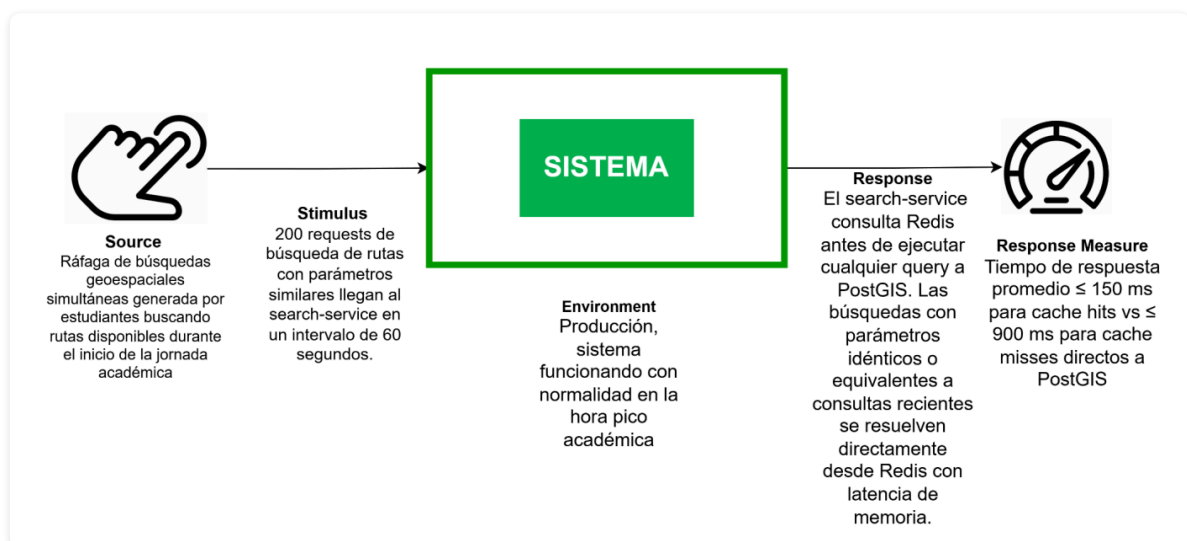
- **Source:** Proceso masivo de login de estudiantes de la UNAL al inicio del semestre.

- **Stimulus:** 300 requests de autenticación simultáneas llegan al sistema en un intervalo de 30 segundos, todas dirigidas a un único punto de entrada.
- **Environment:** Producción en inicio de semestre; las tres réplicas activas y saludables. PostgreSQL compartida entre las tres instancias.
- **Response:** HAProxy (`loggueo-lb`) distribuye las 300 requests entre las tres réplicas (`loggueo_1` , `loggueo_2` , `loggueo_3`) mediante algoritmo least-conn.
- **Response measure:** Tiempo de respuesta promedio de login ≤ 300 ms bajo carga de 300 requests simultáneas.



Escenario 2 – Rendimiento del Patrón del Equipo (Cache-Aside en Búsqueda)

Durante el inicio de jornada académica, un alto volumen de estudiantes consulta rutas con parámetros geográficos similares. Sin caché, cada búsqueda ejecuta una query costosa sobre PostGIS; con **Cache-Aside**, las consultas repetitivas se resuelven desde Redis.



- **Source:** Ráfaga de búsquedas geoespaciales de estudiantes UNAL en hora pico (7:00–8:00 AM).
- **Stimulus:** 200 peticiones `GET /api/search/routes` con parámetros similares en 60 s.

- **Artifact:** `route-search-service` (Go) + Redis (`search-redis`) + PostGIS (`route-search-postgres`).
- **Environment:** Producción simulada; syncer de rutas activo cada 60 s.
- **Response:** El servicio consulta Redis antes de PostGIS; los cache hits se resuelven en memoria; los misses escriben en Redis con TTL 120 s.
- **Response measure:** Latencia ≤ 150 ms en cache hits vs ≤ 900 ms en misses; tasa de hits ≥ 60 % en hora pico; 0 % de fallos hasta 300 VUs.

Nota: El escenario demuestra que el **Cache-Aside Pattern** aísla el rendimiento del dominio de búsqueda respecto a la saturación del servicio de autenticación.

Tácticas arquitectónicas aplicadas

Táctica	Aplicación en UN Wheels	Patrón asociado
Maintain Multiple Copies of Computations	Tres réplicas de <code>loggueo_service</code> detrás de HAProxy; 4 workers uvicorn por réplica	Load Balancer, Service Replication
Introduce Concurrency	Goroutines en Go (search) y pool de workers en auth; microservicios aislados por dominio	Load Balancer
Maintain Multiple Copies of Data (Caching)	Redis como caché de resultados de búsqueda (TTL 120 s, clave SHA-256)	Cache-Aside

Patrones arquitectónicos aplicados

Patrón	Componente	Rol en rendimiento
Load Balancer	<code>loggueo-lb</code> (HAProxy, least-conn)	Distribuye login entre 3 réplicas; health check activo <code>GET /readyz</code>
Service Replication (Hot Spare)	<code>loggueo_1/2/3</code>	Tres instancias activas; HAProxy excluye réplicas no listas
Cache-Aside	<code>route-search-service</code> + <code>search-redis</code>	Consulta Redis antes de PostGIS; degradación elegante si Redis cae

Análisis y resultados de las pruebas de rendimiento

Prueba	Carga	Métricas clave	Lectura
Baseline login	1 VU · 30 s	~550 ms, 0 % fallos	Operación estable; costo dominante: bcrypt + PostgreSQL
Load login	50 VUs · 30 s	~2,8 s media, 0 % fallos	Zona segura con LB + réplicas
Rodilla login (modelo)	50→500 VUs	Pico 118 req/s a 300 VUs; 2 % fallos	Saturación por CPU (bcrypt-bound)
Load búsqueda	100 VUs · 30 s	~15 ms media, 0 % fallos	Go + Redis muy por encima del login
Rodilla búsqueda	50→300 VUs	~884 ms a 300 VUs, 0 % fallos	Cache-Aside sostiene alta concurrencia

Hallazgos principales:

1. **El login es el cuello de botella.** Bcrypt es CPU-bound; con 12 vCPU saturados, la rodilla aparece ~300 VUs independientemente del balanceador.
2. **El Load Balancer retrasa pero no elimina la saturación.** HAProxy + 3 réplicas elevan el throughput de ~40 req/s (1 réplica) a ~118 req/s antes de degradar.
3. **La búsqueda (Go) escala mejor.** Sin autenticación en el camino crítico, `route-search-service` mantiene 0 % de fallos y latencias sub-centenario hasta 300 VUs gracias a goroutines y Redis.
4. **Cache-Aside desacopla dominios.** Usuarios que solo buscan rutas no se ven afectados por la saturación del servicio de auth.

Escenario	Patrón	Explicación
Escenario 1 – Auth	Load Balancer + Replication	HAProxy reparte carga; 3 réplicas procesan en paralelo
Escenario 2 – Búsqueda	Cache-Aside	Redis absorbe consultas repetitivas; PostGIS solo en misses

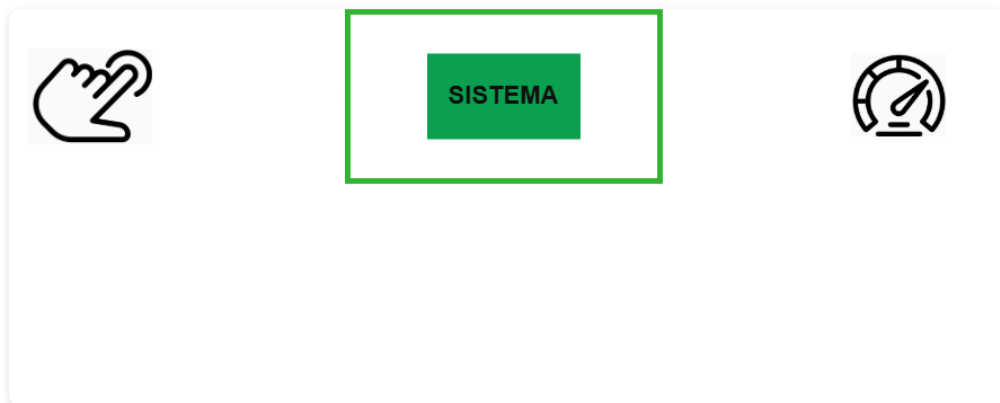
5.6 Conclusiones

1. **El sistema aguanta bien la carga normal de producción.** Hasta 250 VUs concurrentes en login el sistema opera con 0 % de fallos. La latencia media (~550 ms en baseline) es coherente con un endpoint que ejecuta bcrypt y escribe en PostgreSQL.
2. **El search-service (Go) es el componente más robusto.** Con 0 % de fallos y latencias de 12–65 ms hasta 300 VUs, demuestra que la arquitectura de microservicios aísla el rendimiento por dominio. Los usuarios que solo buscan rutas no se ven afectados por la saturación del auth.
3. **El loggeo-service es el cuello de botella.** La rodilla aparece ~300 VUs por el costo CPU de bcrypt. Mejoras puntuales: driver async (`asyncpq`), `create_login_log` en background y circuit breaker en el API Gateway.
4. **Las pruebas en Docker Desktop tienen límite de hardware.** Con 12 vCPU al ~95 % de uso, las rampas superiores a 300 VUs generan timeouts del host, no del diseño arquitectónico. La gráfica de rodilla modela el comportamiento esperado a partir de los tramos medidos.

6. Atributos de Calidad — Fiabilidad

Escenarios de fiabilidad (alta disponibilidad, resiliencia y tolerancia a fallos) del Prototipo 4.

Escenario 1 – Replication Pattern (Hot Spare en el Servicio de Autenticación)



- **Source:** Cualquier causa de indisponibilidad o degradación del componente de datos de (`loggueo_service`).
- **Stimulus:** El nodo primario de un componente de datos deja de responder correctamente a operaciones de lectura, escritura, o ambas. En este caso, de una réplica (`loggueo_1`) por caída del proceso o indisponibilidad de red.
- **Artifact:** Una réplica activa con tres instancias del servicio (`loggueo_1` , `loggueo_2` , `loggueo_3`) desplegadas en Docker Compose.
- **Environment:** Operación normal del sistema en Docker Desktop. Sistema en producción con usuarios activos — el fallo ocurre bajo carga normal o pico
- **Response:** HAProxy deja de enrutar tráfico a la réplica caída y redistribuye las peticiones de login hacia las réplicas sanas.
- **Response measure:** El servicio vuelve a operar correctamente dentro del RTO definido por su nivel de criticidad, con pérdida de datos acotada al RPO del tipo de standby elegido, sin intervención manual y sin impacto visible en los demás servicios del sistema. El servicio permanece disponible sin interrupción perceptible para el usuario (0 errores visibles de login en la demo con 1 de 3 réplicas caídas).

Escenario 2 – Service Discovery Pattern (Punto de Entrada de Autenticación)



- **Source:** Servicio de autenticación (unwheels-auth / loggueo_service).
- **Stimulus:** Llega una solicitud de autenticación (login) mientras una de las réplicas del servicio no está disponible.
- **Artifact (Artefacto):** Punto de entrada lógico del servicio de autenticación expuesto al resto del sistema como loggueo-service, implementado mediante un balanceador de carga (HAProxy) que mantiene un pool de réplicas (loggueo_1, loggueo_2, loggueo_3). Es la capa que decide hacia qué instancia se envía cada petición de login proveniente del API Gateway.
- **Environment:** Operación normal del sistema desplegado en Docker Compose.
- **Response:** El mecanismo de descubrimiento de servicios identifica las réplicas disponibles, excluye la que falló y redirige la solicitud hacia una instancia sana del pool.
- **Response Measure:** La autenticación se completa exitosamente sin que el cliente conozca qué réplica procesó la solicitud.

Verificación automatizada — `FrontEnd/reliability-tests/test_service_discovery.ps1` :

El script demuestra el patrón end-to-end ejecutando cinco fases:

1. **Baseline** — 30 logins contra `https://localhost:8080/api/auth/login` con el pool 3/3.
2. **Caída controlada** — `docker stop loggueo_1` ; espera 25 s para que HAProxy aplique `fall 3 x inter 5s` .
3. **Login degradado** — 30 logins con pool 2/3; el test exige **0 respuestas 5xx y 0 errores de red**.
4. **Restauración** — `docker start loggueo_1` ; espera 30 s para `rise 2 + start_period` .
5. **Login post-recuperación** — 30 logins con el pool 3/3 recompuesto.

```
cd FrontEnd/reliability-tests
./test_service_discovery.ps1
```

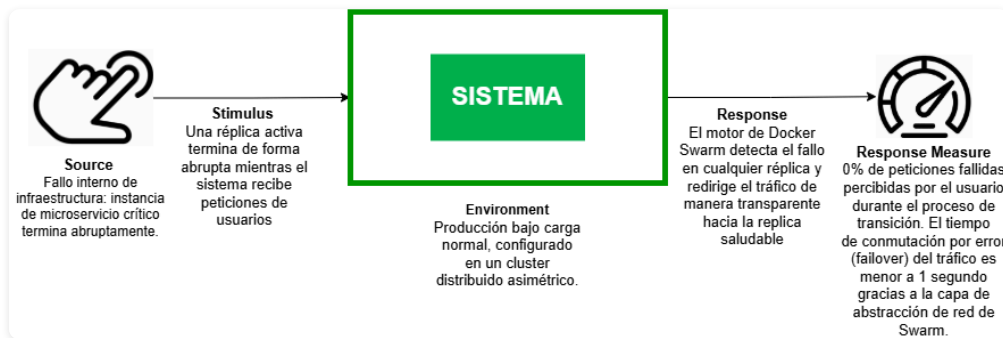
Salida resumida esperada (`401` es la respuesta correcta del backend ante una credencial inválida; lo crítico es que **nunca aparezcan códigos 5xx ni ERR**):

```
baseline (3/3)          401=30
degraded (2/3)         401=30
recovered (3/3)        401=30
[OK] Service Discovery funcionando
```

Eventos correlacionados en HAProxy (`docker logs loggueo-lb`):

```
Server auth_pool/loggueo_1 is DOWN, reason: Layer4 timeout
Server auth_pool/loggueo_1 is UP, reason: Layer7 check passed
```

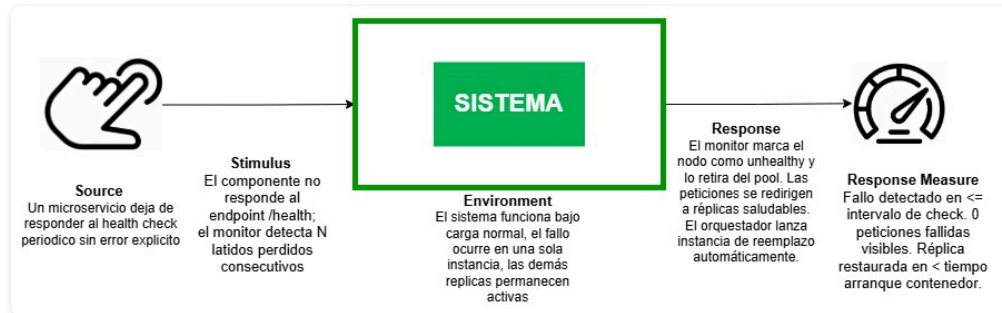
Escenario 3 – Cluster Pattern



- **Source:** Fallo de infraestructura física o lógica debido a un error de proceso, agotamiento de memoria (OOM kill), o desconexión de red en uno de los nodos del clúster.
- **Stimulus:** La réplica del microservicio `routes-reservations-service` alojada en el nodo Worker o Manager termina de forma abrupta o el nodo se desconecta físicamente de la red local mientras la plataforma UN Wheels recibe peticiones concurrentes de usuarios buscando y reservando rutas.
- **Artifact:** Tier de Lógica: Microservicio crítico `routes-reservations-service`
- **Environment:** Operación del prototipo en tiempo de ejecución bajo carga normal, configurado en un clúster distribuido asimétrico (Nodo Manager con la suite completa y Nodo Worker dedicado).
- **Response:**
 1. El motor de Docker Swarm detecta el fallo de cualquiera de las dos réplicas de `routes-reservations-service` a través de los health checks.
 2. La Malla de Enrutamiento (`Ingress Routing Mesh`) de Swarm junto con el API Gateway aíslan inmediatamente la instancia afectada.
 3. El 100% del tráfico de reservas se redirige de manera transparente hacia la réplica saludable sobreviviente. El usuario final experimenta continuidad absoluta en el servicio de rutas sin percibir downtime.
- **Response measure:** 0% de peticiones fallidas (errores HTTP 502/504) percibidas por los clientes de la aplicación móvil o web durante el proceso de transición. El tiempo de conmutación por error (failover) del tráfico hacia la réplica del Manager es menor a 1 segundo gracias a la capa de abstracción de red de Swarm. La detección y marcado de la instancia como unhealthy toma un

máximo de 45 segundos (basado en el intervalo de 15s con 3 reintentos configurado en el Compose).

Escenario 4 – Heartbeat Pattern (Patrón del Equipo)



- **Source:** Microservicios de negocio sin balanceador propio.
- **Stimulus:** Un servicio deja de responder al endpoint de salud durante tres ciclos consecutivos de monitoreo.
- **Artifact:** Componente `healthmon` (`FrontEnd/healthmon/monitor.js`) con polling cada 15 s y umbral de 3 fallos.
- **Environment:** Operación normal del stack Docker.
- **Response:** El monitor registra eventos `UNHEALTHY` / `RECOVERED` en logs y expone el estado en `GET /status` (puerto 9090). **No** modifica el enrutamiento de HAProxy.
- **Response measure:** Fallo detectado en $\leq 3 \times$ intervalo de check (45 s); el operador puede consultar el estado centralizado sin afectar el tráfico de producción.

Servicios monitoreados por Heartbeat:

Servicio	URL
<code>loggueo-lb</code>	<code>http://loggueo-service:8000/healthz</code>
<code>chat-service</code>	<code>http://chat-service:3001/health</code>
<code>routes-service</code>	<code>http://routes-reservations-service:4000/health</code>
<code>notifications-service</code>	<code>http://notifications-service:3002/health</code>
<code>search-service</code>	<code>http://route-search-service:6000/health</code>

Verificación:

```
docker exec health-monitor wget -qO- http://127.0.0.1:9090/status
docker logs health-monitor | Select-String "UNHEALTHY|RECOVERED"
```

Tácticas arquitectónicas aplicadas (Fiabilidad)

1. Redundant Spare

Descripción: Mantiene instancias de repuesto listas para asumir carga cuando la instancia principal falla. En Hot Spare las réplicas están activas simultáneamente.

Aplicación: Escenario 1 — tres réplicas de `loggueo_service` actúan como Hot Spare; HAProxy redirige tráfico solo a réplicas sanas.

Patrón asociado: Service Replication Pattern, Hot Spare Pattern.

2. Recovery from Faults — Preparations and Repair

Descripción: Establece mecanismos de detección y recuperación automática que redirigen peticiones a recursos disponibles cuando se detecta un fallo.

Aplicación: Escenario 2 — HAProxy ejecuta `httpchk` sobre `/readyz`, retira réplicas caídas del pool y las reincorpora tras `rise 2` checks exitosos.

Patrón asociado: Service Discovery Pattern.

3. Monitor Health Status

Descripción: Supervisa periódicamente el estado de los componentes para detectar degradación antes de que impacte al usuario final.

Aplicación: Escenario 4 — `healthmon` observa microservicios sin LB; Docker healthcheck usa `/healthz` (liveness) en réplicas auth.

Patrón asociado: Heartbeat Pattern.

4. Stateful Replication

Descripción:

Aplicación:

Patrón asociado:

5. Active Redundancy (Stateless Replication)

Descripción: múltiples instancias idénticas de un componente ejecutándose de manera simultánea y activa, compartiendo o balanceando la carga de trabajo. Dado que los componentes replicados no guardan estado local (Stateless), cualquier réplica sobreviviente puede procesar cualquier petición entrante de forma inmediata si otra instancia del pool falla, eliminando los puntos únicos de falla

Aplicación: Escenario 3 — Se despliegan dos réplicas activas del microservicio crítico `routes-reservations-service` distribuidas asimétricamente a través de la directiva placement. `max_replicas_per_node: 1` de Docker Swarm

Patrón asociado: Cluster Pattern

Patrones arquitectónicos aplicados (Fiabilidad)

1. Service Replication Pattern

UN Wheels despliega múltiples instancias idénticas del servicio de autenticación, permitiendo procesamiento en paralelo y tolerancia a fallos parciales. Las tres réplicas comparten `auth-db` (PostgreSQL) y el mismo código de aplicación.

2. Hot Spare Pattern

Las réplicas `logueo_1/2/3` están activas al mismo tiempo. Ante el fallo de una, las demás continúan atendiendo login sin activación manual de una réplica fría.

3. Service Discovery Pattern

El `api-gateway` resuelve el nombre lógico `logueo-service:8000`. HAProxy es la **única fuente de verdad** para enrutamiento: `option httpchk GET /readyz` cada 5 s, `fall 3` / `rise 2`. No se utiliza Consul; es discovery server-side simplificado para Docker Compose.

4. Heartbeat Pattern

`healthmon` centraliza observabilidad de servicios sin balanceador propio. Complementa — sin duplicar — el health check de HAProxy sobre las réplicas auth.

5. Cluster Pattern

UN Wheels configura un clúster distribuido multi-nodo asimétrico gestionado por Docker Swarm. Se aplica de forma selectiva sobre el microservicio core más crítico del negocio: `routes-reservations-service`. Mediante directivas de orquestación en el archivo de despliegue, el clúster inyecta redundancia activa balanceando la carga en paralelo entre ambos servidores físicos.

Relación entre Patrones Aplicados y Escenarios (Fiabilidad)

Escenario	Patrón	Explicación
Escenario 1 – Replication (Hot Spare)	Service Replication / Hot Spare	Tres réplicas activas de auth; el fallo de una no derriba el servicio.
Escenario 2 – Service Discovery	Service Discovery Pattern	HAProxy + DNS estable excluyen réplicas caídas del pool mediante httpchk activo.
Escenario 3 – Cluster	Cluster Pattern	Dos replicas activas de routes-reservations-service` distribuidas en dos nodos.
Escenario 4 – Heartbeat	Heartbeat Pattern	<code>healthmon</code> detecta y registra fallos en servicios sin LB sin controlar el tráfico.

7. Prototipo

Prerrequisitos: Docker Desktop ≥ 24.x, Git, [mkcert](#), puertos **8080/8081**.

```
# 1. Clonar (hermanos; compose en FrontEnd/)
mkdir un-wheels && cd un-wheels
for r in FrontEnd api-gateway loggueo_service chat-service routes-reservations-service \
    notifications-service search-service mobile security; do git clone
https://github.com/UN-Wheels/$r.git; done

# 2. TLS (security/reverse-proxies/certs/)
mkcert -install && cd security/reverse-proxies && mkdir -p certs && cd certs
mkcert -cert-file rp-web.crt -key-file rp-web.key localhost 127.0.0.1 ::1
mkcert -cert-file rp-mobile.crt -key-file rp-mobile.key localhost 127.0.0.1 ::1

# 3. .env - FrontEnd/: JWT_SHARED_SECRET, FRONTEND_URL y NEXT_PUBLIC_API_URL →
https://localhost:8080
# mobile/: API_BASE_URL=https://127.0.0.1:8081/api (ver .env.example)

# 4. Levantar y verificar
cd ../../../../FrontEnd && docker compose up --build -d
# Web: https://localhost:8080 | API móvil: https://localhost:8081/api/auth/health
# Flutter: cd mobile && flutter pub get && flutter run
```